Which Parts of a Tool's Documentation Does a Model Actually Need?

A controlled ablation of tool schemas on BFCL and ToolSandbox multi-turn tasks

Randeep Singh
Lucio AI
randeep@lucioai.com

November 2026

Abstract

A function-calling interface documents every tool through four pieces of prose, a tool name, a tool description, parameter names and parameter descriptions, alongside the structural fields of its JSON Schema. We ask which of the four a language model actually depends on when it works through a stateful, multi-turn task. Holding the Berkeley Function Calling Leaderboard (BFCL) multi-turn base tasks, backends and scorer fixed, we remove one component at a time, and then all four, for four models (GPT-6 Luna, Claude Haiku 5.5, Qwen3-30B-A3B and Gemma 4 26B-A4B) on the same 200 tasks, with paired statistics and a repeated baseline that measures run-to-run noise. Parameter descriptions are the only component whose removal hurts every model: task success falls by 14 to 33 percentage points, while removing tool descriptions or tool names stays within noise. The recorded trajectories locate the damage at argument construction rather than tool selection and trace it to a handful of parameters whose descriptions were the only place an allowed-value list, a value format or an entity type was stated. Writing those constraints into the schema as `enum`, `pattern` and better parameter names, with every description left blank, restores baseline success for all four models. With all documentation removed, success is at most 1% under BFCL's evaluator and no model recovers a task after an execution error. On a second benchmark, Apple's ToolSandbox, whose names, descriptions and parameter names each restate the others, removing any one component costs nothing and removing all four costs 39 to 49 points of similarity. Switching reasoning off for the two models that reason by default, and on for one that does not, leaves every effect in place. The models need each constraint on a tool's use stated once, in any carrier; parameter descriptions mattered on BFCL because they were the carrier that happened to hold the constraints.

1 Introduction

Function calling is now the standard way to connect a language model to software. The model is shown a list of tools, each described by a JSON Schema (Wright et al., 2022) with a name, a natural-language description, and for every parameter a name, a type, optional structural constraints and a further description; it answers a user by emitting calls against that list, reads the results, and continues (Anthropic, 2024; OpenAI, 2023). The interface has become the unit of integration for agents, and the prose inside it has become a design surface in its own right: practitioners write and rewrite tool descriptions, and a line of work studies how to write them well (Guo et al., 2026; Hsieh et al., 2023; Qu et al., 2025a; Yuan et al., 2025). What is missing is controlled evidence about which parts of that prose a model relies on once it is inside a realistic task, with state to track, several turns to get through and a few dozen tools to choose from.

This paper provides such evidence. We take the 200 multi-turn base tasks of the Berkeley Function Calling Leaderboard (BFCL) (Mao et al., 2024; Patil et al., 2025), hold every part of the

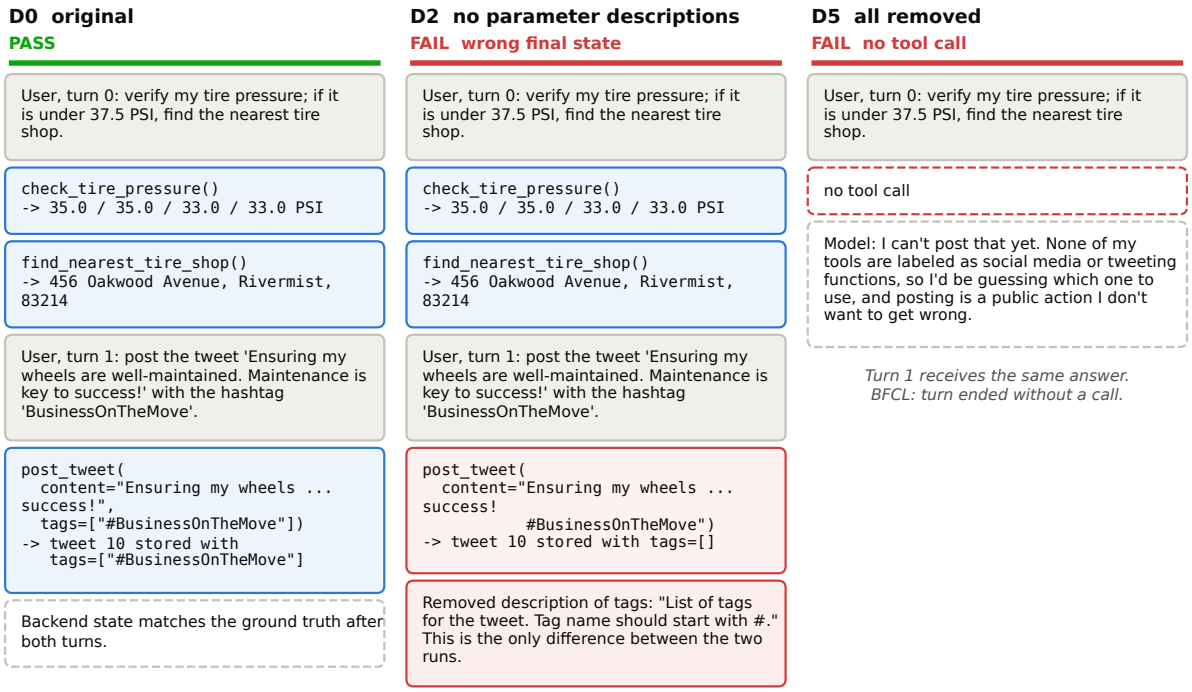


Figure 1: One task, three documentation conditions. Claude Haiku 5.5 on BFCL task `multi_turn_base_78`. With the original schema (D0) the model checks the tires, finds the shop and posts the tweet with the hashtag in the `tags` list. With parameter descriptions removed (D2) the first turn is identical, but the hashtag is folded into the tweet body, the backend stores no tags, and the final state no longer matches the ground truth. With names and descriptions removed (D5) the model declines to guess which tool posts and never acts. Section 5.5 and Appendix C give the full trajectories.

benchmark fixed except the documentation shown to the model, and remove that documentation one component at a time. The four components are the ones every function-calling schema has: tool names, tool descriptions, parameter names and parameter descriptions. Six conditions remove each of the four on its own or all four at once, against the unmodified schema; two follow-up conditions then test what the removed parameter descriptions were contributing. Types, required lists, enums, defaults and the response schema are kept in every condition, as are tool outputs and backend state, so the experiment measures the contribution of the prose rather than of the information an agent can obtain by interacting. Four models from four providers ran every task under every condition. Because the same tasks appear in every condition, every comparison is paired at the task level, and a repeated baseline run measures how much two identical runs disagree. The single-component conditions were then repeated on Apple's ToolSandbox (Lu et al., 2024), which differs from BFCL in its backends, its scorer, in simulating the user with a language model and in documenting every tool redundantly, and the main comparisons were repeated with each model's reasoning switched explicitly on or off.

Section 2 sets up the vocabulary the rest of the paper uses. A tool call involves two decisions, selecting a tool and constructing its arguments, and a schema carries two kinds of information, the meaning of a tool and the constraints on its arguments, through four prose carriers and a set of structural fields. The experiment is an ablation of the carriers; the analysis asks which decision each carrier serves, and whether the information it carried was available anywhere else. Because every raw request and response, every tool call, every execution result and the backend state after every turn were recorded, the trajectories can answer that question directly. Our findings are:

- **Parameter descriptions are the component the models depend on** (Section 4). Removing them (D2) costs 14.0 to 33.0 percentage points for every model, significant after

Bonferroni correction. Removing tool descriptions (D1) and replacing tool names with `tool_NN` (D3) stay within the noise band of a repeated baseline for every model. Replacing parameter names with `arg_N` (D4) is harmless for GPT-6 Luna and Gemma 4, costs Qwen3 11.5 points, and produces an apparent gain for Claude Haiku that disappears against the repeated baseline and with reasoning off.

- **The damage is at argument construction, not tool selection** (Section 5.1). In D2 the right tool is chosen for the first call as often as in D0, but the right tool with the right arguments falls by 11 to 15 points, and failures begin with a wrong-argument call rather than a missing tool.
- **The effect sits on a handful of prose-only constraints** (Section 5.2). Six parameters account for most of the new argument errors in every model; their descriptions were the only place an allowed-value list (`order_type`: "Buy/Sell"), a format convention (`tags` start with #) or an entity type the name hides (`cityA` is a zip code) was stated. The D2 drop on the 91 tasks that use them is 36 to 70 points; on the other 109 tasks it is within ±5 points.
- **Structure substitutes for the prose** (Section 5.6). Keeping every parameter description blank but writing the prose-only constraints into `enum`, `pattern` and two better parameter names (D6) brings all four models back to baseline. Keeping only the six descriptions (D7) recovers most of the loss.
- **Where the semantics live does not matter, as long as they are somewhere** (Section 4.5). On ToolSandbox, whose four carriers each restate the others, removing any one leaves every model within a few points of baseline and the right tools are still called; removing all four costs 39 to 49 points.
- **Reasoning does not change the effect** (Section 4.6). With the same weights and reasoning switched off (Luna, Haiku) or on (Gemma 4), the D2 drop stays within 5.5 points of the default run for every pair. Models reason two to eight times longer when documentation is removed and recover none of the loss.
- **Nobody compensates through exploration** (Section 5.3). With everything removed (D5), success is at most 1% under BFCL's 20-step, final-state evaluator; two models stop calling tools, one probes blindly to the step limit, and none recovers a task after an execution error.

The rest of the paper is organised as follows. Section 2 gives the background on tool calling and the definitions on which the study rests. Section 3 describes the benchmarks, the conditions, the models and the metrics. Section 4 reports the main results, Section 5 analyses the trajectories, and Section 6 interprets the findings. Section 7 places the study in the tool-learning literature, Section 8 states its limitations, and Section 9 concludes.

2 Background: tool calling

This section fixes the terms used in the rest of the paper. It describes the interface through which a model is shown its tools (Section 2.1), the loop in which it uses them and the way stateful benchmarks score that loop (Section 2.2), and the two decisions a model makes at every call and the information each decision needs (Section 2.3). The conditions of Section 3 are defined as operations on the interface of Section 2.1, the metrics of Section 3.4 as statistics of the loop of Section 2.2, and the analysis of Section 5 as a test of which decision of Section 2.3 each component serves.

2.1 The function-calling interface

Function calling, "also called tool use, refers to an LLM's ability to invoke external functions, APIs, or user-defined tools" (Patil et al., 2025). Early tool-augmented models inserted calls into free text, as Toolformer does with special tokens (Schick et al., 2023) or ReAct with a fixed action format (Yao et al., 2023), and were trained or prompted for a small, fixed set of tools. Models trained on large API collections (Patil et al., 2023; Qin et al., 2024) made the tool set an input rather than a constant, and provider-native function calling (OpenAI, 2023) standardised how that input is expressed: each tool is a JSON object with a `name`, a `description` and a `parameters` field holding a JSON Schema (Wright et al., 2022) for its arguments. The Model Context Protocol (Anthropic, 2024) adopts the same object as the unit of exchange between tool servers and models, so the format is now shared across providers and frameworks. BFCL distinguishes models that receive the tools through this native interface ("FC mode") from models prompted to emit calls as text ("prompting mode") (Patil et al., 2025); every model in this study runs in the former, so what the model sees is exactly the serialised schema and nothing else describes the tools.

Formally, a tool is a tuple $\tau = (n, d, P)$ consisting of a name n, a description d and a parameter specification P. Each parameter $p \in P$ has a name, a type, a set of structural fields and a description. The structural fields are the ones JSON Schema defines with a fixed meaning: whether the parameter is `required`, its `default`, an `enum` of admissible values, a `pattern` a string must match, and for object-typed parameters a nested specification of the same form. Many benchmarks and APIs also attach a `response` schema describing the return value, with its own descriptions. A model is shown a tool set $\mathcal{T} = \{\tau_1, \ldots, \tau_K\}$, typically between ten and forty tools, serialised in the provider's format, and is trained to answer with either text or one or more calls (n, a), where n is the name of a tool in $\mathcal{T}$ and a is a JSON object of argument values that should conform to that tool's P.

Two kinds of information are thus mixed in one object. The *structural fields* (types, required lists, enums, patterns, defaults, nesting) are machine-readable and have one meaning. The *prose carriers* are four: the tool name n, the tool description d, the parameter names and the parameter descriptions. Names are short and conventionally descriptive (`post_tweet`, `phone_number`); descriptions are free text and may say anything, including things the structural fields could have said. A description such as "Type of order (Buy/Sell)" states an admissible-value set, and "Tags for the tweet, e.g. #tag" states a format, in a place where no parser enforces them. We call a constraint on an argument value *structural* if a structural field expresses it and *prose-only* if it is stated only in a description. Whether a model treats a prose-only constraint differently from a structural one is one of the questions the study answers.

2.2 The interaction loop and its evaluation

BFCL formalises a single-turn instance as a triple (q, F, A) of a user query, a function set and the labelled call (Patil et al., 2025), and τ-bench formalises a multi-turn task as a partially observable Markov decision process whose actions are tool calls and user messages (Yao et al., 2025). The description here stays closer to the transcript. A tool-calling conversation alternates between a user and the model. Within a user *turn* the model takes one or more *steps*: at each step it is sent the conversation so far, including every earlier call and result, and the tool set, and it replies with calls or with text. Each call is executed against a backend, which returns a result (or an error) that is appended to the conversation before the next step; a text reply ends the turn. Backends may be stateless, as in a calculator or a search engine, or *stateful*, as in a file system or a booking system, in which case a call can change the state that later calls and later turns see. Stateful, multi-turn tasks are the setting in which documentation can matter most and in which it can be compensated for: a wrong call produces an observable error or a wrong intermediate result, and the model has further steps in which to react.

How such a conversation is scored determines what an experiment on it can measure. Single-call benchmarks such as API-Bank (Li et al., 2023) and the single-turn categories of BFCL (Patil et al., 2025; Yan et al., 2024) compare each emitted call with a reference call, by abstract-syntax-tree matching or by execution. Stateful benchmarks instead compare outcomes. BFCL's multi-turn categories (Mao et al., 2024) run the model against deterministic Python backends, apply a ground-truth sequence of calls to a copy of the same backends, and pass a turn only if the two final states agree and any required outputs were produced; τ-bench (Yao et al., 2025) scores the final database state against a reference and simulates the user with a language model; ToolSandbox (Lu et al., 2024) also simulates the user, and scores a trajectory by how closely its best-matching points meet a set of milestones. Outcome-based scoring is what allows a documentation ablation to be isolated: the model may reach the required state by any sequence of calls, so the score reflects what it achieved rather than whether it reproduced a reference transcript, and the backend, the user turns and the ground truth are identical in every condition.

2.3 Two decisions and what they need

At every step the model that decides to act makes two decisions: it *selects* a tool $n \in \mathcal{T}$, and it *constructs* an argument object a for that tool. Surveys of tool learning decompose the process into planning, selection, calling and response generation (Qu et al., 2025b); the ablation here does not manipulate the user's request, so planning is held fixed and the two decisions that depend on the schema are selection and construction. Selection needs the model to know what each tool *does*, well enough to pick the one that serves the user's request from the K on offer. Construction needs the model to know what each parameter *is*, which parameters are required, and which values are admissible: the constraints of Section 2.1, structural or prose-only, plus the values themselves, which come from the user's request, from earlier results or from convention.

Each piece of that information may be stated in several carriers at once. A tool's purpose can be read from its name, from its description, and often from its parameter list; a parameter's meaning from its name or its description; a value's admissible set from an `enum` or from a description. Where two carriers state the same thing, removing one leaves the other, and a model that uses either should be unaffected. Where one carrier is the only place a piece of information is stated, removing it removes the information, and the model must either infer it from the rest of the interface, discover it by interacting, or fail. The decomposition yields a prediction for each ablation: removing a carrier should cost nothing if its content is redundant, and should cost at the decision that needs the content if it is not. The decomposition also says what to look for in the trajectories. A selection failure shows as the right tool never being called; a construction failure shows as the right tool being called with wrong values; and an interactive recovery shows as a later call that corrects the earlier one. The experimental setup that follows ablates the four prose carriers one at a time; the analysis of Section 5 classifies every failure by the decision at which it occurred and every new argument error by the constraint that was removed.

3 Experimental setup

The study is an ablation of the four prose carriers of Section 2.1, run on stateful multi-turn tasks scored by outcome (Section 2.2), and measured both by task success and by statistics of the loop that separate the two decisions of Section 2.3. This section describes the two benchmarks, the conditions, the models and the harness, and the metrics.

3.1 Benchmarks

BFCL multi-turn base. BFCL's `multi_turn_base` category (Mao et al., 2024) contains 200 hand-curated tasks. Each task is a conversation of one to seven user turns (median four) over a small set of stateful Python backends: a file system, a vehicle-control API, a trading bot, a travel

Table 1: Documentation conditions. "Generic" tool names are `tool_01`...`tool_NN` in BFCL's listing order; generic parameter names are `arg_1`...`arg_n` per tool, applied recursively to object-typed parameters. Parameter types, required lists, enums, defaults and the response-schema structure are kept in every condition. †D1 also removes the natural-language descriptions inside the response (return-value) schema, so it is an ablation of all tool-level prose, not of the one-line tool description alone. ‡D6 renames `cityA`/`cityB` to `zipcodeA`/`zipcodeB`; all other names are original. D6 and D7 were run after the main study, in response to its results.

	Tool names	Tool descriptions	Parameter names	Parameter descriptions
D0	original	original	original	original
D1	original	**removed**†	original	original
D2	original	original	original	**removed**
D3	**generic**	original	original	original
D4	original	original	**generic**	original
D5	**generic**	**removed**	**generic**	**removed**
D6	original	original	original‡	**removed**; constraints in schema
D7	original	original	original	**removed**; six kept (Table 7)

API, a messaging API, a Twitter-like API, a ticketing API and a math API. A task involves one or two backends, and the model is shown every function of the involved backends, 18 to 39 tools per task (median 28), as the tool set $\mathcal{T}$ of Section 2.1. The ground truth lists, for each turn, the minimal set of calls that bring the backend into the required state (2 to 10 calls per task, mean 5.7). A turn is scored by BFCL's multi-turn checker as described in Section 2.2: the backend state after the turn must match the state produced by the ground-truth calls (ignoring private attributes), and for read-only requests the model's execution path must contain the ground-truth outputs; a task passes only if every turn passes. Within a turn the model may take up to 20 steps; exceeding the limit terminates the task as a failure. We used BFCL at commit `6ea57973` (Gorilla LLM contributors, 2026) with its default temperature of 0.001, and did not modify the harness, the backends or the evaluator.

ToolSandbox. The single-component conditions were also run on Apple's ToolSandbox (Lu et al., 2024), used unmodified at commit `c8571d7`, which differs from BFCL in every respect that could carry a documentation effect. The user is simulated on-policy by a language model (GPT-4o-2024-05-13, as in the original paper), so the agent can ask for clarification and the conversation is not scripted; the tools are Python functions over a phone-like world state (contacts, messages, reminders, device settings, time and unit utilities) whose schemas are generated from their docstrings, so every description is a short "Args" line rather than BFCL's long prose; and the score is a similarity in $[0, 1]$ between the trajectory and the scenario's milestones (exact match on identifiers, ROUGE-L on free text) rather than a binary verdict. We run the 91 base scenarios that need no paid web API, each with 24 to 28 tools on offer, for the same four models, applying the conditions to the generated schemas exactly as for BFCL and translating the model's calls back before the sandbox executes them. Appendix B gives the remaining details of the setup and every result.

3.2 Documentation conditions

Each condition is an operation on the tool tuple (n, d, P) that leaves the structural fields untouched (Table 1). D1 blanks the tool description d; because BFCL's response schemas also describe what a tool does, D1 blanks their descriptions too, keeping their structure and types. D2 blanks every parameter description (blanked rather than deleted, because BFCL's schema converter expects the key to exist). D3 replaces every tool name n with a generic `tool_NN` in listing order, and D4 replaces every parameter name with a generic `arg_N`, recursively through object-typed parameters. D5 applies all four at once. In conditions with generic names the

model's calls are translated back to the original tool and parameter names before BFCL executes and scores them, so the backend and the evaluator see exactly what they would see in D0; the chat history the model sees keeps the names it was given. Names the model invents are passed through untouched and fail inside BFCL's executor as they would in the unmodified benchmark. In D0 the tools sent to the model are byte-identical to a stock BFCL run against an OpenAI-compatible endpoint.

In the terms of Section 2.3, D1 to D4 each remove one carrier while the other three remain, so each measures what that carrier contributed beyond what the rest of the interface, the user's request and the tool outputs supply. D5 removes all four carriers, leaving only the structural fields, and is a test of *reduced* interface semantics rather than of zero semantics: the user's request, the tool outputs, the parameter types, requiredness and enum values all remain, and tools remain distinguishable by their argument structure.

D6 and D7 are follow-ups designed after the main analysis identified six parameters as the carriers of the D2 effect (Section 5.2). Both blank every parameter description exactly as D2 does. D6 then converts every prose-only constraint of the BFCL schemas into a structural one: every parameter whose description contains an allowed-value list ("[Enum]: ..." or "(Buy/Sell)") receives a JSON Schema `enum` with those values (11 parameters across the vehicle, trading and travel backends), `post_tweet.tags` and `mentions` receive an item `pattern` of `^#` and `^@`, and `estimate_distance.cityA`/`cityB` are renamed `zipcodeA`/`zipcodeB`. D7 instead keeps the original descriptions of the six parameters of Table 7 and blanks all others. D6 measures whether a structural carrier can replace the prose one; D7 measures how much of the D2 drop those six descriptions alone account for.

3.3 Models and harness

We evaluated four models through OpenRouter (OpenRouter, 2026), chosen to span two frontier small models and two open-weight mixture-of-experts models of similar size: GPT-6 Luna (OpenAI, 2026), Claude Haiku 5.5 (Anthropic, 2026), Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025) and Gemma 4 26B-A4B (Google DeepMind, 2026). All four were used in native function-calling mode, so they received the tool set in the format of Section 2.1 as serialised by their provider. Provider routing was left to OpenRouter. Every Luna request was served by Azure and every Haiku request by Google; Gemma 4 was served by DeepInfra except for 7 of 1 200 trajectories with some requests routed to Makora; Qwen3 was served by SiliconFlow in every trajectory, but 336 of its 1 200 trajectories had at least one request routed to Nebius. Qwen3's success is lower on those mixed-provider trajectories in every condition (for example 47% against 57% in D0 and 17% against 34% in D2), so the Qwen3 numbers should be read as those of a two-provider deployment; the D2 effect is present on both subsets.

No reasoning-effort parameter was set in the main study: 99% of Luna and Haiku trajectories contain reasoning tokens (674 and 664 per trajectory on average), while Qwen3 emitted none and Gemma 4 did so in 7 of its 1 200 trajectories (at most 280 tokens). Any contrast between the frontier and the open-weight pair is therefore confounded with reasoning; the within-model comparisons that carry the paper's claims are not. To test whether reasoning changes the documentation effect itself, three reasoning-controlled pairs were run after the main study (Section 4.6): the same model id with OpenRouter's `reasoning` parameter set explicitly, off for Luna and Haiku (which reason by default) and on for Gemma 4 (which does not), on all 200 tasks under D0 to D5. Qwen3-30B-A3B-Instruct-2507 has no reasoning switch (its thinking variant is a different checkpoint), so it has no pair.

The harness subclasses BFCL's OpenAI-compatible handler. It differs from a stock BFCL run only in routing requests to OpenRouter, applying the condition to the tool schemas, translating generic names back before execution, and recording one JSON file per trajectory containing the task, the exact tools sent, the name maps, BFCL's result and inference log (with tool outputs and backend state after every turn), every raw HTTP request and response, and the evaluator's

verdict. The main study (4 models × 6 conditions × 200 tasks = 4 800 trajectories) ran in about two hours of wall-clock time on 2 November 2026 with eight concurrent tasks per model-condition pair and four pairs in parallel; the two follow-up conditions (1 600 trajectories) and a repeated D0 run of Claude Haiku 5.5 (200 trajectories) ran the same afternoon with the same settings. The reasoning-controlled pairs ($3 \times 6 \times 200 = 3\,600$ trajectories) and the ToolSandbox runs ($4 \times 6 \times 91 = 2\,184$ plays, sixteen in parallel) ran on 3 November 2026. Four Qwen3 trajectories (one each in D0 and D3, two in D5) ended in a provider error and are counted as failures; dropping the three affected tasks pairwise instead changes no paired difference by more than 0.8 pp and no significance verdict.

3.4 Metrics and statistics

Task success is BFCL's own verdict and is the primary metric. The behavioural metrics are statistics of the loop of Section 2.2, computed from the trajectory files and the ground truth without any model-based judging. A *call* is a tool call BFCL decoded from the model's response; malformed calls end the turn and are not counted. A *step* is one model request; a plain-text reply also counts as a step and ends the turn. A call is *on-path* if its tool appears anywhere in the task's ground truth, *off-path* if it does not but executed without error, and *failed* if the backend returned an execution error. A *repeated* call has identical tool and arguments to an earlier call in the same task.

The two decisions of Section 2.3 are measured against the ground truth as follows. For each ground-truth call in turn t, a *tool hit* means the model called that tool in any step of turns 0 to t (matching is cumulative across the task, not per turn), and an *exact hit* means one of those calls had identical arguments. Two argument values match if they are equal, if both are numeric and differ by less than 10^{-9}, or if they are equal as whitespace-stripped strings; there is no case folding, so `"buy"` does not match `"Buy"`. One model call can satisfy several identical ground-truth calls, and each ground-truth call is credited at most once. *Tool-selection accuracy* is tool hits over all ground-truth calls and measures the selection decision; *argument accuracy given the right tool* is exact hits over tool hits and measures the construction decision conditional on selection having succeeded. Because an exact hit requires every argument value to match, a semantically equivalent but differently spelled value counts as a miss; this makes argument accuracy conservative in every condition equally. For each failed task the *first divergence* is the earliest ground-truth call, in turn order, without an exact hit, classified as *wrong arguments* if the tool was called with other arguments (a construction failure) or *tool never called* otherwise (a selection failure). When a ground-truth call was matched by the right tool but wrong arguments, the closest same-tool call is found and the parameters whose values differ are recorded; counting these in D2 and subtracting the D0 count isolates the errors that appear when parameter descriptions are removed, and links each to the description that was removed. *Recovery rate* is the share of tasks containing at least one failed call that nevertheless passed, and measures interactive recovery.

Because every task appears in every condition, each ablation is compared with D0 by a paired analysis (McNemar, 1947): we report the two discordant cells (tasks passing only in D0 and tasks passing only in the ablation), the exact two-sided McNemar p-value computed from them, and a 95% confidence interval for the paired success-rate difference from a task-level bootstrap with 2 000 resamples (Efron, 1979). With 20 pre-planned comparisons (five ablations × four models) we use a Bonferroni threshold of $p < 0.0025$ (Dunn, 1961) and treat everything else as exploratory; the follow-up comparisons in Section 5.6 are reported with the same test but were not part of the pre-planned family.

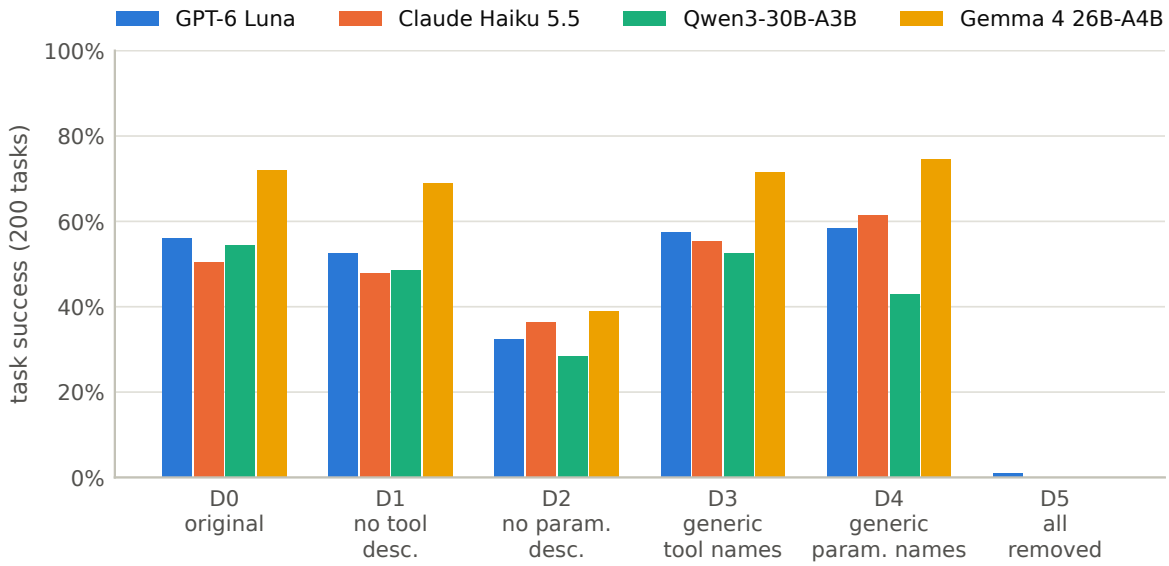


Figure 2: Task success rate by documentation condition. Each bar is 200 tasks; the same tasks appear in every bar, so bars within a model are paired.

Table 2: Task success (%) per condition, 200 tasks per cell.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna	56.0	52.5	32.5	57.5	58.5	1.0
Claude Haiku 5.5	50.5	48.0	36.5	55.5	61.5	0.0
Qwen3-30B-A3B	54.5	48.5	28.5	52.5	43.0	0.0
Gemma 4 26B-A4B	72.0	69.0	39.0	71.5	74.5	0.0

4 Results

4.1 Documentation sensitivity

Removing parameter descriptions is the only single ablation that hurts every model. Figure 2 and Table 2 give the headline result, and every model shows the same shape: a modest dip for D1, a large drop for D2, no change for D3, and a collapse for D5. Gemma 4 has the highest baseline (72.0%) and the largest absolute D2 drop (33.0 pp). The heatmap in Figure 3 shows the same numbers as paired changes. A single-number sensitivity score per model is given in Appendix A (Table 10) but, because D2 dominates every model's sensitivity, it conceals more than it shows.

4.2 Paired task-level statistics

D2 and D5 are significant for every model; D1 and D3 are not. Table 3 gives the paired tests. D2 and D5 are significant for all four models at the corrected threshold. The D2 discordant cells are lopsided: tasks that pass only in D0 outnumber tasks that pass only in D2 by 53 to 6 (Luna), 42 to 14 (Haiku), 63 to 11 (Qwen3) and 71 to 5 (Gemma 4). D1 and D3 have discordant cells of similar size in both directions for every model, which is what run-to-run noise at near-zero temperature looks like; their confidence intervals all include zero. The two D4 results that survive correction point in opposite directions: Haiku gains (29 tasks pass only in D4 against 7 only in D0; $p = 0.0003$) and Qwen3 loses (12 against 35; $p = 0.0011$). Haiku's 29 gained tasks are not concentrated anywhere: they span 16 combinations of backends, 19 of them involve the six parameters of Table 7 and 10 do not, and their D0 failures begin with a wrong argument in 12 cases, a never-called tool in 7 and no identifiable divergence in 10. Section 4.3

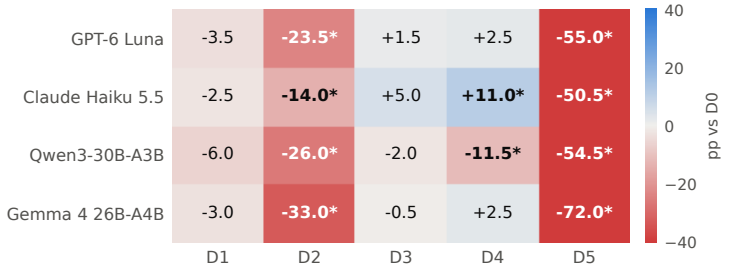


Figure 3: Change in task success relative to D0, in percentage points. Starred, bold cells are significant at the Bonferroni-corrected threshold $p < 0.0025$ (exact McNemar test, Table 3). Colour saturates at ± 40 pp.

Table 3: Each ablation versus D0 on the same 200 tasks. The two discordant cells carry the evidence; Δ is the paired success-rate difference in percentage points with a 95% task-level bootstrap interval (2 000 resamples); p is the exact two-sided McNemar test. The last column applies the Bonferroni threshold for 20 comparisons.

Model	Cond.	both pass	D0 only	abl. only	both fail	Δ pp	95% CI	McNemar p	$p < 0.0025$
GPT-6 Luna	D1	88	24	17	71	-3.5	[-9.5, 2.5]	0.3489	no
	D2	59	53	6	82	-23.5	[-30.0, -17.0]	<0.0001	**yes**
	D3	97	15	18	70	+1.5	[-4.0, 7.5]	0.7283	no
	D4	98	14	19	69	+2.5	[-3.0, 8.0]	0.4869	no
	D5	2	110	0	88	-55.0	[-61.5, -48.0]	<0.0001	**yes**
Claude Haiku 5.5	D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224	no
	D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002	**yes**
	D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742	no
	D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003	**yes**
	D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001	**yes**
Qwen3-30B-A3B	D1	78	31	19	72	-6.0	[-13.0, 0.5]	0.1189	no
	D2	46	63	11	80	-26.0	[-34.0, -18.5]	<0.0001	**yes**
	D3	85	24	20	71	-2.0	[-8.5, 4.5]	0.6516	no
	D4	74	35	12	79	-11.5	[-18.5, -5.0]	0.0011	**yes**
	D5	0	109	0	91	-54.5	[-61.5, -48.0]	<0.0001	**yes**
Gemma 4 26B-A4B	D1	130	14	8	48	-3.0	[-7.5, 1.5]	0.2863	no
	D2	73	71	5	51	-33.0	[-40.5, -26.0]	<0.0001	**yes**
	D3	136	8	7	49	-0.5	[-4.5, 3.5]	1.0000	no
	D4	141	3	8	48	+2.5	[-0.5, 5.5]	0.2266	no
	D5	0	144	0	56	-72.0	[-78.5, -66.0]	<0.0001	**yes**

puts this result against a direct measurement of run-to-run noise.

4.3 Run-to-run noise

Two identical runs differ by as much as D1 or D3. The D1 and D3 discordant cells suggested that two runs of the same condition differ on roughly 15 to 30 tasks in each direction. Table 4 measures this directly: Claude Haiku 5.5's second D0 run passed 56.5% of tasks against 50.5% in the first, with 9 tasks passing only in the first run and 21 only in the second ($\Delta = +6.0$ pp, $p = 0.0428$). At temperature 0.001 the models are not deterministic, the backends are, so every discordance here is the model's. The gap between the two identical runs is as large as the D1 and D3 gaps against either of them, and is itself nominally significant at $p < 0.05$ (though not at the corrected threshold). Three consequences follow. The D1, D3 and (for three models) D4 differences are of the same size as this floor and should be read as null results. The D2 and D5 differences are an order of magnitude larger than it and remain significant against the second D0 run as well as the first (Haiku's D2 drop is 14.0 pp against the first run and 20.0 pp against the second). Haiku's D4 gain, +11.0 pp and significant against the first D0 run, shrinks to +5.0 pp against the second ($p = 0.13$), with the two D0 runs themselves 6 points apart. We

Table 4: A repeated D0 run of Claude Haiku 5.5 against the first, and each ablation against both D0 runs. Same 200 tasks, same settings, run eight hours apart. The discordance between two identical runs is the floor against which single-condition differences must be read.

Comparison	both pass	first only	second only	both fail	Δ pp	95% CI	McNemar p
D0 run 1 vs D0 run 2	92	9	21	78	+6.0	[1.0, 11.5]	0.0428
D0 run 1 vs D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224
D0 run 2 vs D1	86	27	10	77	-8.5	[-14.5, -3.0]	0.0076
D0 run 1 vs D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002
D0 run 2 vs D2	61	52	12	75	-20.0	[-27.0, -13.0]	<0.0001
D0 run 1 vs D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742
D0 run 2 vs D3	97	16	14	73	-1.0	[-6.0, 4.5]	0.8555
D0 run 1 vs D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003
D0 run 2 vs D4	100	13	23	64	+5.0	[-1.0, 11.0]	0.1325
D0 run 1 vs D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001
D0 run 2 vs D5	0	113	0	87	-56.5	[-63.0, -49.5]	<0.0001
D0 run 1 vs D6	87	14	21	78	+3.5	[-2.0, 9.5]	0.3105
D0 run 2 vs D6	95	18	13	74	-2.5	[-8.0, 3.0]	0.4731
D0 run 1 vs D7	85	16	25	74	+4.5	[-2.0, 11.0]	0.2110
D0 run 2 vs D7	96	17	14	73	-1.5	[-7.0, 3.5]	0.7201

report it as suggestive at most and offer no mechanism for it; with Haiku's reasoning switched off it disappears entirely (Section 4.6).

4.4 Memorisation check

No model recalled an original name. BFCL's multi-turn tasks predate all four models, so a model could in principle recognise the backends from their argument structure and call the original tool names even when shown `tool_NN`. Invented names pass through the harness untouched and would appear in the raw responses. Across the 1 515 to 3 927 tool calls each model made in D3 and D5, no model emitted a single original tool name, and across D4 and D5 no model emitted a single original parameter name; the only non-generic strings were eight malformed generic names from Qwen3 (`tool_6` for `tool_06`) and ten non-generic argument keys from Luna in D4 and D5. The D3 and D5 results therefore measure what the models do with the interface they were shown, not what they remember.

4.5 A second benchmark: where the semantics live does not matter

When every carrier restates the others, none can be shown to matter and all of them together can. ToolSandbox (Section 3) is the case Section 2.3 describes as fully redundant. Its tool names are full phrases (`set_wifi_status`, `search_messages`), its tool descriptions paraphrase the name ("Add a new contact person to contact database"), its parameter names paraphrase their descriptions (`phone_number`: "Phone number of contact person"), and none of its 72 parameter descriptions carries a constraint that the schema lacks. The prediction is that no single carrier can be shown to matter, and none can: D1, D2, D3 and D4 each leave every model within a few points of its baseline similarity (D2 moves it by −6.3 to +2.0 points, significant for no model), and the ablated run still calls every tool that the baseline run used in 66 to 91% of the scenarios the baseline solved. Removing all four at once (D5) costs every model 39 to 49 points ($p < 0.0001$) and that share falls to 0 to 13%. The models therefore do not depend on any particular carrier of a tool's semantics, only on the semantics being stated in at least one of them. The BFCL D2 effect is the case where one carrier was the only one, and D6 (Section 5.6) the case where the same semantics were moved into another. Appendix B reports the ToolSandbox setup in full, the results, the paired tests, a per-category breakdown and the behaviour under D5.

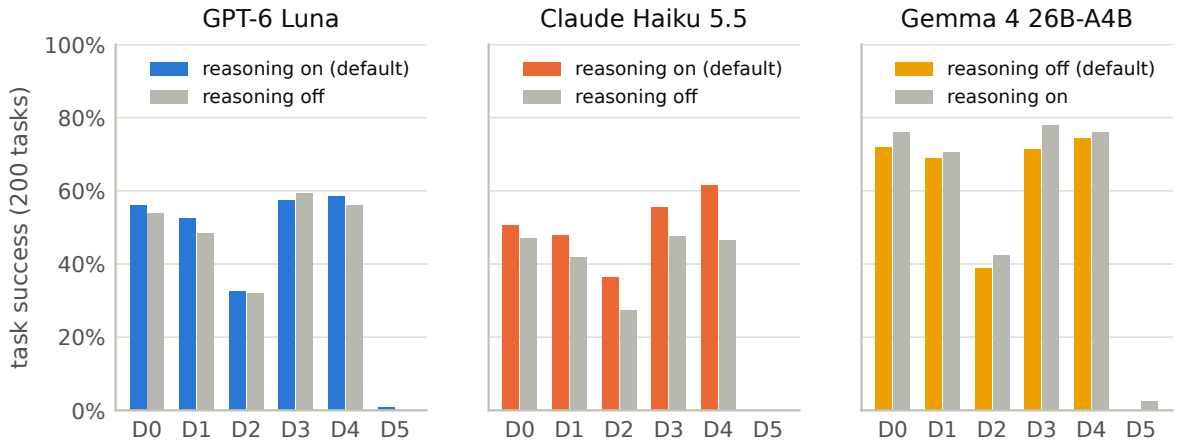


Figure 4: Task success by condition for the three reasoning-controlled pairs: the same model id with OpenRouter's reasoning switch explicitly off (Luna, Haiku) or on (Gemma 4), against the default run of the main study. The shape of the documentation effect is the same in every pair.

Table 5: Reasoning-controlled pairs on BFCL. *: the model's default, i.e. the main-study run. "Tokens" is the mean number of reasoning tokens per task at D0. ΔD2 and ΔD4 are the paired differences from that variant's own D0 (percentage points, 95% bootstrap CI, exact McNemar p).

			task success (%)									
Model	Reasoning	Tokens	D0	D1	D2	D3	D4	D5	ΔD2 [95% CI]	p	ΔD4 [95% CI]	p
GPT-6 Luna	on*	330	56.0	52.5	32.5	57.5	58.5	1.0	-23.5 [-30.5, -16.5]	<0.0001	+2.5 [-3.0, 8.0]	0.4869
	off	0	54.0	48.5	32.0	59.5	56.0	0.0	-22.0 [-30.0, -14.0]	<0.0001	+2.0 [-5.0, 8.5]	0.6655
Claude Haiku 5.5	on*	517	50.5	48.0	36.5	55.5	61.5	0.0	-14.0 [-21.0, -7.0]	0.0002	+11.0 [5.5, 16.5]	0.0003
	off	0	47.0	42.0	27.5	47.5	46.5	0.0	-19.5 [-26.0, -13.0]	<0.0001	-0.5 [-6.5, 5.5]	1.0000
Gemma 4 26B-A4B	off*	0	72.0	69.0	39.0	71.5	74.5	0.0	-33.0 [-40.0, -26.0]	<0.0001	+2.5 [-0.5, 6.0]	0.2266
	on	1110	76.0	70.5	42.5	78.0	76.0	2.5	-33.5 [-41.0, -26.5]	<0.0001	+0.0 [-2.5, 2.5]	1.0000

4.6 Reasoning-controlled pairs

Three of the four models can be run with the same weights and reasoning explicitly switched (Section 3): Luna and Haiku reason by default and were rerun with reasoning off, Gemma 4 does not and was rerun with reasoning on, each on all 200 tasks under D0 to D5. The switch took: with reasoning off, Luna and Haiku emitted no reasoning tokens in any of their 1 200 trajectories; with it on, Gemma 4 emitted 1 110 per task at D0. Figure 4 and Table 5 give the results.

Reasoning does not change the documentation effect. The D2 drop is −23.5 pp with Luna's reasoning on and −22.0 pp with it off, −14.0 and −19.5 pp for Haiku, −33.0 and −33.5 pp for Gemma 4; all six are significant at the corrected threshold, and the D1 and D3 nulls hold in every variant. D5 success is at most 2.5% in every variant; Gemma 4 with reasoning on passes five D5 tasks, the most of any run, at 8 659 reasoning tokens per task.

Reasoning changes the level, and Haiku's D4 result. Turning reasoning off costs Luna nothing measurable at any condition (paired differences of −4.0 to +2.0 pp, $p \geq 0.29$). It costs Haiku between 3.5 and 15.0 pp at every condition, significantly at D2 (−9.0 pp, $p = 0.0009$) and D4 (−15.0 pp, $p = 0.0001$). Haiku's D4 gain of Section 4.3, +11.0 pp with reasoning on, is −0.5 pp with it off ($p = 1.0$): whatever produces it belongs to Haiku's reasoning mode, not to the generic parameter names, and it is no longer a candidate for a documentation effect. Turning reasoning on raises Gemma 4 by 1.5 to 6.5 pp at every condition but D5, nominally significant only at D3 (+6.5 pp, $p = 0.024$), and leaves its D2 drop where it was.

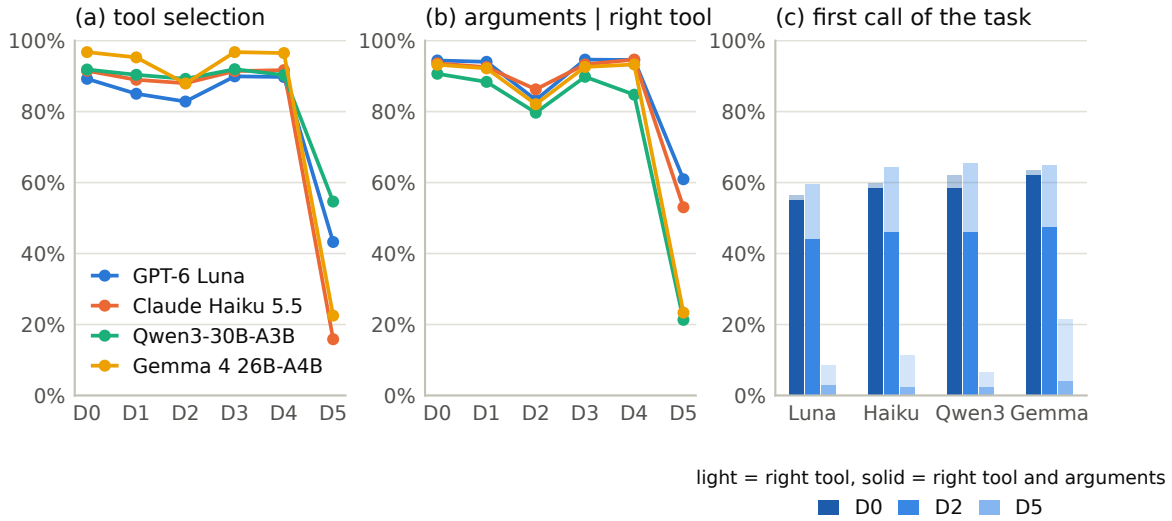


Figure 5: (a) Tool-selection accuracy: share of ground-truth calls whose tool the model invoked by the end of the required turn. (b) Argument accuracy conditional on the right tool: share of tool hits that were also exact-argument hits. (c) First call of the task: share of tasks whose first call used a ground-truth tool (light) and used it with exactly the ground-truth arguments (solid), for D0, D2 and D5.

Table 6: Tool selection versus argument construction (%). "Sel." is tool-selection accuracy, "Arg." argument accuracy given the right tool, "1st tool" the share of tasks whose first call used a ground-truth tool, and "1st exact" the share whose first call matched a ground-truth call exactly.

	D0				D2				D5			
Model	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact
GPT-6 Luna	89.2	94.4	56.5	55.0	82.8	83.3	59.5	44.0	43.3	60.9	8.5	3.0
Claude Haiku 5.5	91.5	93.6	60.0	58.5	88.0	86.3	64.5	46.0	15.8	53.0	11.5	2.5
Qwen3-30B-A3B	91.9	90.7	62.0	58.5	89.2	79.7	65.5	46.0	54.6	21.3	6.5	2.5
Gemma 4 26B-A4B	96.8	93.2	63.5	62.0	87.9	82.1	65.0	47.5	22.5	23.3	21.5	4.0

Models think longer when the documentation says less, and recover nothing by it. The reasoning budget responds to the interface even though success does not: per task, Luna spends 330 reasoning tokens at D0, 390 at D2 and 2 294 at D5; Haiku 517, 559 and 1 258; Gemma 4 with reasoning on 1 110, 2 554 and 8 659. The models think longer when the documentation says less, and recover none of the loss by doing so.

5 Analysis: where documentation matters

The aggregate results say that parameter descriptions matter. Section 2.3 says where they could matter: at the selection decision, at the construction decision, or in interactive recovery. This section uses the recorded trajectories to tell the three apart, identifies the constraints whose removal produced the new errors, and closes with the two follow-up conditions that test the resulting explanation.

5.1 Selecting tools versus constructing arguments

D2 leaves the selection decision intact and damages the construction decision. Figure 5 and Table 6 separate the two with the metrics of Section 3.4. Under the single ablations tool-selection accuracy is nearly flat: between D0 and D2 it moves by 2.6 to 8.8 points, and part of even that movement is a cascade effect, because a turn derailed by a wrong argument may never reach its later ground-truth tools. Argument accuracy given the right tool drops in D2 for

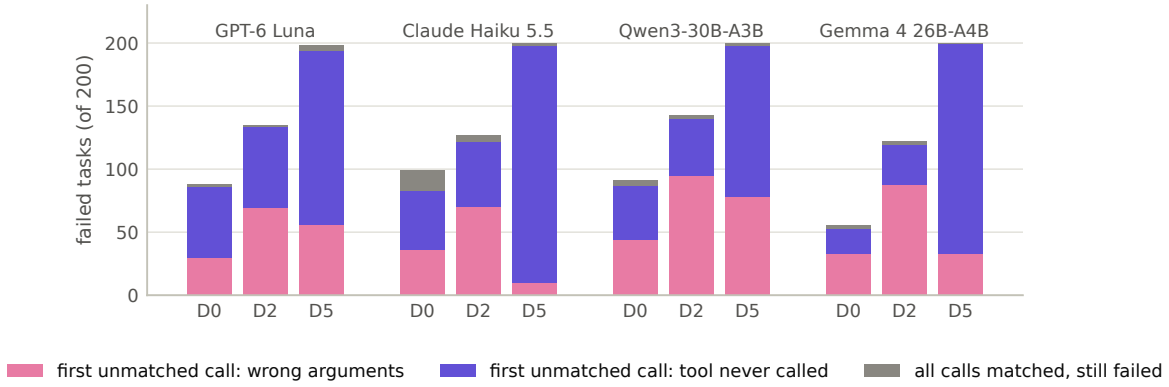


Figure 6: Earliest divergence in failed tasks. For each failed task, the first ground-truth call (in turn order) without an exact match is classified by whether the tool was called with wrong arguments or never called at all; "all calls matched" means every ground-truth call was matched at some point but the task still failed, usually because of extra state-changing calls or a call made in a later turn than required.

every model: 94.4% → 83.3% (Luna), 93.6% → 86.3% (Haiku), 90.7% → 79.7% (Qwen3) and 93.2% → 82.1% (Gemma 4).

The first call shows it without cascade effects. The first call of each task, which is free of cascade effects, tells the story most cleanly (Figure 5c). In D2 the right tool is chosen for the first call slightly *more* often than in D0 for every model (56.5% → 59.5%, 60.0% → 64.5%, 62.0% → 65.5%, 63.5% → 65.0%), but the right tool with the right arguments falls by 11.0, 12.5, 12.5 and 14.5 points. The models know what to call; they no longer know what to put in it.

D2 failures begin with a wrong argument; D5 failures begin with a missing tool. Figure 6 (and Table 11 in the appendix) classify each failed task by its first divergence. Among D2 failures the earliest divergence is a wrong-argument call rather than a missing tool for 69 of 135 (Luna), 70 of 127 (Haiku), 95 of 143 (Qwen3) and 88 of 122 (Gemma 4) tasks. Compared with D0, the number of failures that begin with a wrong-argument call rises by 39, 34, 51 and 55, whereas the number that begin with a never-called tool rises by only 9, 5, 2 and 11. D5 is different in kind: tool selection itself collapses (43.3%, 15.8%, 54.6%, 22.5%) and most failures begin with a tool that was never called, which is what Section 2.3 predicts once every carrier of a tool's purpose is gone.

5.2 Which descriptions matter

Six parameters carry the effect, and each was the only carrier of a constraint. If the construction decision is what D2 damages, the trajectories should show which values went wrong and the removed descriptions should show why. Table 7 links each new argument error to the description that was removed. Six parameters account for most of the new errors, and they are the same six for every model. What their descriptions have in common is that each was the only carrier of a constraint, in the sense of Section 2.1: a prose-only constraint that no structural field and no name expressed.

- an allowed-value list written only in prose: `place_order.order_type` ("Buy/Sell") and `startEngine.ignitionMode` ("[Enum]: START, STOP"), both plain string fields with no schema `enum`;
- an entity type that differs from what the parameter name suggests: `estimate_distance.cityA` and `cityB` take zip codes, not city names;

Table 7: Parameters whose values go wrong when their description is removed. For every ground-truth call matched by the right tool but wrong arguments, the closest same-tool call was found and the differing parameters recorded. Counts are pooled over the four models (800 task runs per condition), sorted by the D2−D0 difference; per-model D2 counts on the right. The second column quotes the description that D2 removed.

Tool.parameter	Removed description	D2	D0	Luna	Haiku	Qwen3	Gemma
`place_order.order_type`	*Type of the order (Buy/Sell).*	**108**	0	26	27	27	28
`startEngine.ignitionMode`	*The ignition mode of the vehicle. [Enum]: ["START", "STOP"]*	**97**	0	25	14	28	30
`estimate_distance.cityA`	*The zipcode of the first city.*	**75**	4	20	20	16	19
`estimate_distance.cityB`	*The zipcode of the second city.*	**75**	4	20	20	16	19
`post_tweet.tags`	*List of tags for the tweet. Tag name should start with #. This is only relevant if the user wants to add tags to the tweet.*	**72**	3	22	18	12	20
`post_tweet.mentions`	*List of users mentioned in the tweet. Mention name should start with @. This is only relevant if the user wants to add mentions to the tweet.*	**45**	4	15	11	8	11
`post_tweet.content`	*Content of the tweet.*	**37**	26	6	17	3	11
`get_zipcode_based_on_city.city`	*The name of the city.*	**10**	0	1	4	4	1
`compute_exchange_rate.base_currency`	*The base currency. [Enum]: USD, RMB, EUR, JPY, GBP, CAD, AUD, INR, RUB, BRL, MXN*	**8**	0	3	1	2	2
`get_available_stocks.sector`	*The sector to retrieve stocks from (e.g., 'Technology').*	**8**	0	3	0	4	1

- a format convention: `post_tweet.tags` must start with `#` and `mentions` with `@`.

Without the description, models wrote `"buy"` instead of `"Buy"`, `"start"` instead of `"START"`, passed city names where zip codes were required, or folded the hashtag into the tweet body instead of the `tags` list (the `content` row in the table is the mirror image of `tags`: when the hashtag is folded into the content, the content value is also wrong). The last pattern appears in three of the four case studies in Section 5.5. The backends react to these errors in two different ways. `place_order` lower-cases `order_type` for its funds check but stores the value as given, so `"buy"` is accepted silently and only the final-state comparison catches it; `post_tweet` likewise stores the hashtag-laden content with an empty tag list. `startEngine("start")` returns "Invalid ignition mode." and `estimate_distance` with city names returns "distance not found in database.", so for those two parameters the model received an explicit error and, as Section 5.3 shows, usually did not repair the argument.

At task level, the D2 effect is an effect on the tasks that use them. Table 7 counts argument errors, not tasks, so it does not by itself establish how much of the D2 drop the six parameters explain. The ground truth of 91 of the 200 tasks contains at least one of them. Of the tasks that pass in D0 and fail in D2, 44 of 53 (Luna), 37 of 42 (Haiku), 50 of 63 (Qwen3) and 64 of 71 (Gemma 4) are among those 91. Splitting the success rates accordingly (Table 8), the D2 drop on the 91 tasks is 47.3, 36.3, 51.6 and 70.3 pp, while on the other 109 tasks it is +3.7, −4.6, +4.6 and +1.8 pp, within the run-to-run variation measured in Section 4.3. At task-level resolution the D2 effect is an effect on tasks that use these six parameters; Section 5.6 shows that a tail of other prose-only enums adds a few points on top.

This narrows the headline finding. Parameter descriptions matter here less as meaning and more as the place where this benchmark keeps its enums, formats and entity conventions. Section 2.3 predicts that moving a constraint from one carrier to another should change nothing;

Table 8: Success (%) on the 91 tasks whose ground truth uses at least one of the six parameters of Table 7 ("six") and on the other 109 tasks ("other"), per condition. The first column counts the tasks that pass in D0 and fail in D2 and, of those, how many are six-parameter tasks.

	D0-only tasks	D0		D2		D6		D7	
Model	six / all	six	other	six	other	six	other	six	other
GPT-6 Luna	44/53	51.6	59.6	4.4	56.0	50.5	56.9	49.5	54.1
Claude Haiku 5.5	37/42	46.2	54.1	9.9	58.7	49.5	57.8	51.6	57.8
Qwen3-30B-A3B	50/63	60.4	49.5	8.8	45.0	59.3	44.0	49.5	40.4
Gemma 4 26B-A4B	64/71	75.8	68.8	5.5	67.0	75.8	65.1	64.8	64.2

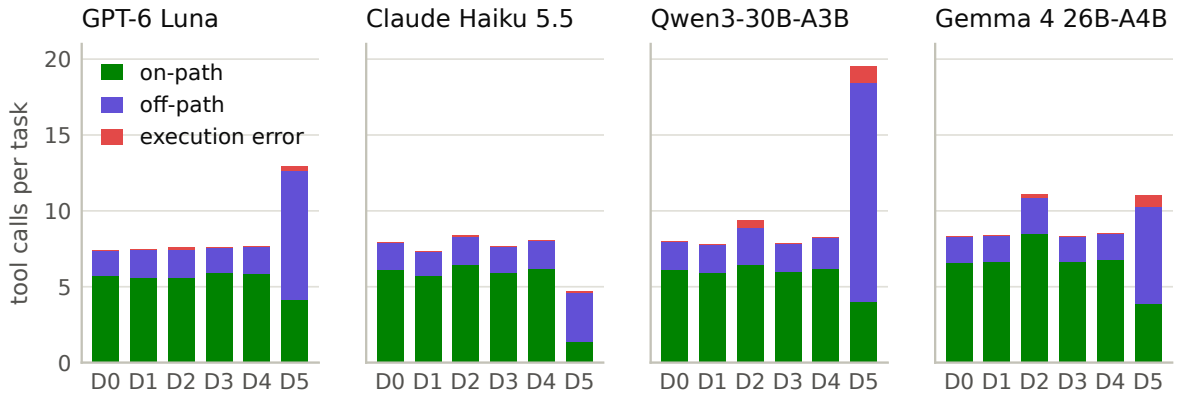


Figure 7: Composition of tool calls per task (mean over all 200 tasks, failures included): on-path calls, off-path calls that executed without error, and calls that returned an execution error.

Section 5.6 tests that prediction with a schema that moves the prose-only constraints into `enum`, `pattern` and precise parameter names while the descriptions stay blank.

5.3 Tool-use behaviour, recovery and exploration

Single ablations do not change how much a model interacts. The third possibility of Section 2.3 is that a model compensates for missing documentation by interacting: calling more, probing, and correcting after errors. Figure 7 (full numbers in Table 13) shows that under the single ablations it does not. D2 adds 0.2, 0.5, 1.4 and 2.8 calls per task over D0 for Luna, Haiku, Qwen3 and Gemma 4 respectively. The exception is Gemma 4, which repeats calls when it is unsure (3.39 repeated calls per task in D2 against 1.24 in D0) and hits the 20-step limit in 20 tasks (4 in D0). The composition shifts rather than the count: failed calls per task rise from at most 0.02 in D0 to between 0.13 and 0.51 in D2.

D5 produces two opposite behaviours. Haiku makes *fewer* calls than in D0 (4.7 against 7.9), does not call any tool in the first turn of 34.5% of tasks, and most of the calls it does make are off-path. Qwen3 makes 19.5 calls per task, touching 15.6 distinct tools of the roughly 28 available, with 1.09 failed calls per task and an execution error in 51% of tasks. Luna sits between (12.9 calls, 9.1 distinct tools, no first-turn call in 22% of tasks), as does Gemma 4 (11.0 calls, 5.0 distinct tools). The ECDFs in Figure 8 show that D2 shifts the call distribution only slightly to the right, while D5 pulls it in opposite directions by model: Haiku's curve rises steeply at 0 to 3 calls, Qwen3's stretches out to the step cap.

Neither restraint nor probing turns into recovered tasks. Recovery after an execution error is poor in D2 and absent in D5 (Figure 9, left; Table 12). In D2, of the tasks that contained an execution error, 12% of 24 (Luna), 22% of 23 (Haiku), 20% of 30 (Qwen3) and 21% of 29

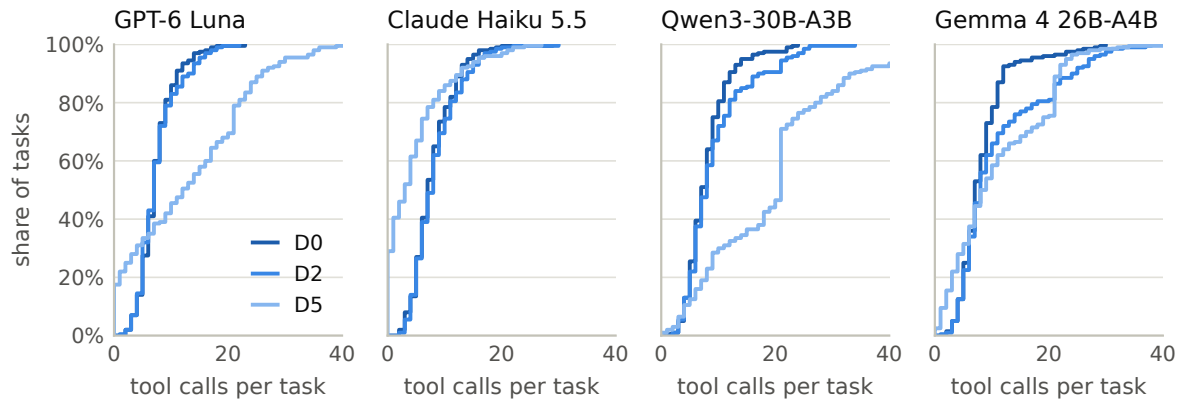


Figure 8: Empirical cumulative distribution of tool calls per task for D0, D2 and D5. All 200 tasks per curve, including failed and force-terminated runs. A curve further to the right means more calls.

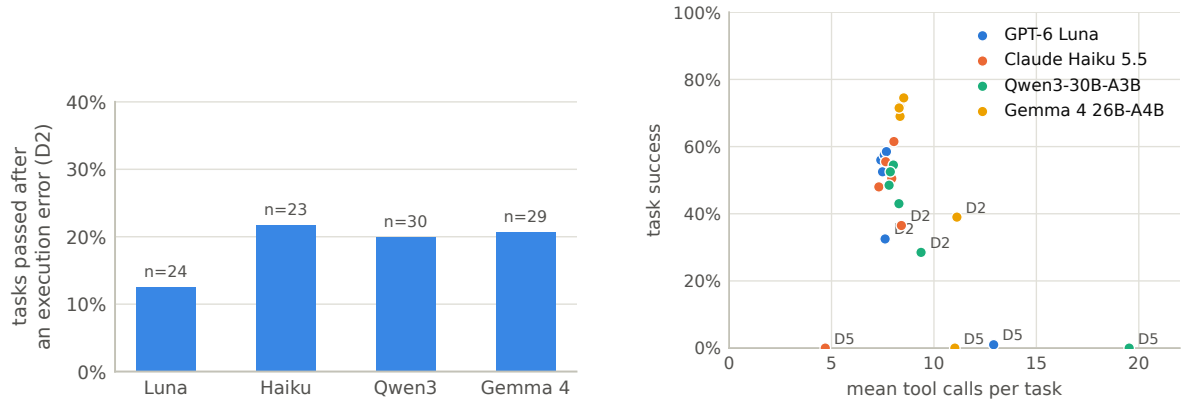


Figure 9: Left: recovery after an execution error in D2: share of tasks containing at least one failed call that still passed, with the number of such tasks. D0 denominators are 1 to 4 tasks and are not comparable; in D5 no model recovered any of its 17 to 102 such tasks (Table 12). **Right:** efficiency frontier, task success against mean tool calls per task, one point per model and condition; D2 and D5 points are labelled.

(Gemma 4) still passed. D0 recovery rates look higher but rest on one to four tasks each. In D5 no model recovered a single task after a failed call: 0 of 38, 17, 102 and 30. The efficiency frontier (Figure 9, right) is flat across single ablations: D0 through D4 sit at 7 to 9 calls per task for every model and the vertical spread between them is the D2 effect. D5 moves points right (Qwen3, Luna, Gemma 4) or left (Haiku) without moving them up.

5.4 Failure outcomes

D2 adds wrong final states; D5 adds silence or exhausted budgets. BFCL assigns one terminal error per failed task (Figure 10, Table 14). In D2 the extra failures are almost all "wrong final state" (61 → 96, 64 → 86, 64 → 97 and 35 → 76 tasks from D0 to D2): the model acted, but on the wrong values. In D5 the mix changes with the model's strategy. Haiku's and Luna's D5 failures are dominated by turns that ended without any tool call (78 and 50 tasks), Qwen3's by hitting the 20-step limit (57 tasks) and Gemma 4's by a wrong final state after probing (94 tasks).

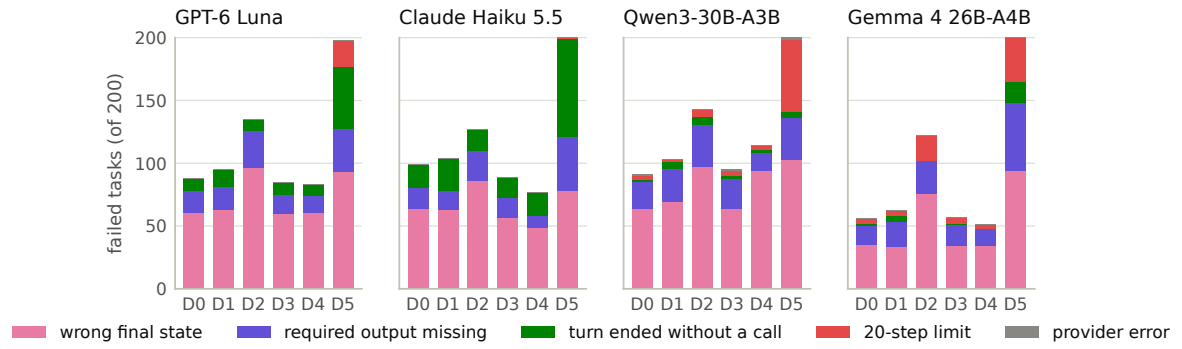


Figure 10: BFCL's terminal error per failed task, by condition. These are observed outcomes, not causes: a wrong final state can come from a wrong tool, a wrong argument or a missed step.

5.5 Case studies

Case studies were selected by a rule applied identically to every model: among tasks that pass in D0 and fail in D2, take the one with the fewest ground-truth calls (lowest task id on ties), and show the same task in D0, D2 and D5. The rule favours short tasks so that the whole trajectory can be shown; it does not select for drama. Figure 1 shows the Claude Haiku case, the other three are summarised below, and Appendix C gives every call of all four cases under the three conditions.

Claude Haiku 5.5, task 78 (Figure 1). Turn 0 is identical in D0 and D2. In turn 1, D0 passes the hashtag as `tags=["#BusinessOnTheMove"]`; D2 folds it into `content`, so the backend records a tweet with no tags and the final state differs. The only information that changed between the two runs is the removed `tags` description quoted in the figure. In D5 the model never calls a tool and tells the user it will not guess which of the unlabeled tools posts to social media; that is a refusal, not an exploration.

Gemma 4 26B-A4B, task 11. The user asks to list the files in a `/temp` directory (which does not exist; the files are in the current directory) and then to tweet their names with the hashtag `#fileshowcase`. Both D0 and D2 wander in turn 0, repeating `ls` and `find` until they settle, and both end turn 0 in the correct state. The outcome is decided in turn 1 by the same hashtag-in-content error as the Haiku case. In D5 the single call is `get_user_stats(username="ls -a /temp")`: the model picked a Twitter tool and passed the shell command it wanted to run as its argument.

GPT-6 Luna, task 177. The user asks to book a business-class flight from Los Angeles to Tokyo and then to "confirm the booking's details". D2 shows a recovered argument error and an unrecovered tool choice. In turn 0 it first calls `get_flight_cost` with city names (the removed descriptions said the parameters take airport codes), gets an execution error, corrects to `LAX` and `HND`, and books correctly. In turn 1, D0 calls `retrieve_invoice` while D2 calls `get_booking_history`. Both return the booking, but BFCL requires the invoice output, so D2 fails with "required output missing". In D5 the model answers turn 0 with an essay about verifying itineraries and never calls a tool.

Qwen3-30B-A3B, task 42. The user asks to create `Notes2023.txt` in an existing `Lectures` folder, write a sentence to it, and count its characters. The divergence is in turn 0. In D0, after `cd Lectures` fails, the model calls `pwd`, discovers it is already inside `/Lectures`, and creates the file. In D2 it stops after the failed `cd` and an empty `ls`, so turn 0 ends with no file created; in

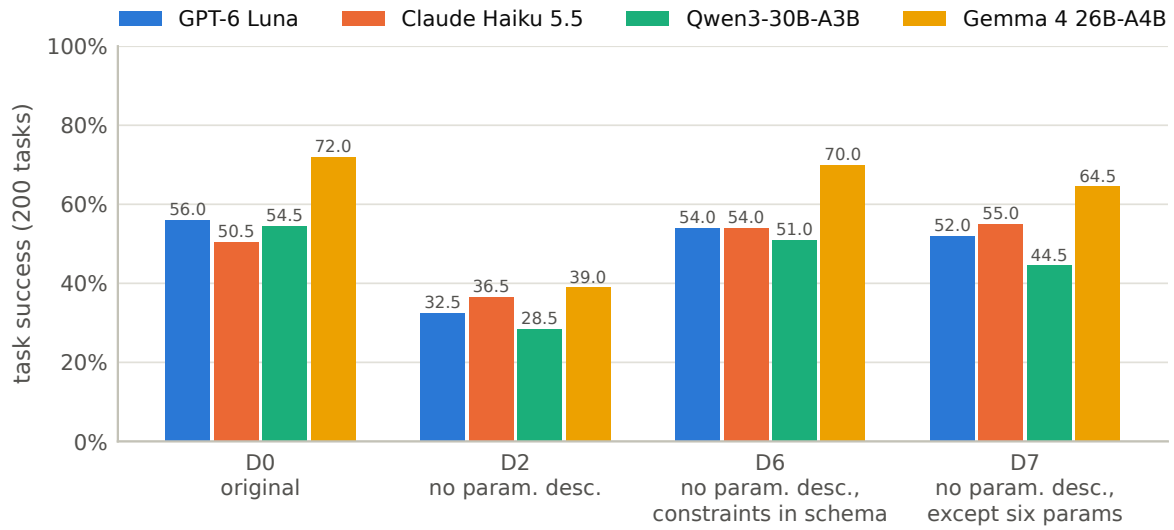


Figure 11: Follow-up conditions. D6 and D7 both blank every parameter description, as D2 does. D6 moves the prose-only constraints into the schema (`enum`, `pattern`, two renamed parameters); D7 keeps the descriptions of the six parameters of Table 7 only. Same 200 tasks as every other bar.

turn 1 it creates a new `Lectures` folder inside the existing one and writes the file there. Every later call succeeds, but the directory tree has an extra folder and BFCL scores a wrong final state. Nothing about the documentation of the tools involved differs between D0 and D2; this is a case where the run diverged on a near-tie decision and the evaluator is strict about state. It is a reminder that not every D0/D2 discordant task is a documentation effect, which is why the paired statistics count both discordant cells. In D5 the model tries eleven different tools with guessed arguments and gives up.

5.6 Follow-up: restoring the structure without the prose

Writing the constraints into the schema restores baseline with every description blank. Section 5.2 argued that the parameter descriptions that matter are the ones that were the only carrier of a constraint the schema could have expressed. If that is right, a schema that expresses the same constraints structurally should perform like D0 with every description blank, and a schema that keeps only the six descriptions should recover most but not all of the loss. Figure 11 and Table 9 test both predictions with the conditions defined in Section 3. In D6, with every description still blank but the prose-only enums moved into `enum`, patterns added for tags and mentions, and the two zip-code parameters renamed, success is 54.0% (Luna), 54.0% (Haiku), 51.0% (Qwen3) and 70.0% (Gemma 4), against 56.0, 50.5, 54.5 and 72.0% in D0: every model is within 3.5 points of its baseline, no D6–D0 difference is significant, and every D6–D2 difference is, recovering 21.5, 17.5, 22.5 and 31.0 points. D7, which restores only the six descriptions, reaches 52.0, 55.0, 44.5 and 64.5%: it recovers most of the D2 loss for every model but stays 4 to 10 points below D0 for three of them (significant for Qwen3 at the uncorrected level), which is what one expects if a few further prose-only constraints outside the six contribute a little each. Table 8 shows the same picture split by task: on the 91 tasks that use the six parameters, D6 is back at D0 for every model, and on the other 109 tasks D2 and D6 stay within the noise band of D0 for every model, while D7 is slightly below D0 for Qwen3 (-9 points). The parameters that go wrong in D7 but not in D6 are exactly the remaining prose-only enums that D6 converts and D7 does not, led by `lockDoors.door` (20 wrong values pooled over the models against 3 in D0) and `activateParkingBrake.mode` (7 against 0).

Table 9: Follow-up conditions against D0 and against D2 on the same 200 tasks: success of the base and follow-up condition, the discordant cells (tasks passing only in the base condition, only in the follow-up), the paired difference with its 95% bootstrap interval, and the exact McNemar p.

Model	Comparison	base %	f.-up %	base only	f.-up only	Δ pp	95% CI	p
GPT-6 Luna	D6 vs D0	56.0	54.0	19	15	-2.0	[-7.5, 4.0]	0.6076
	D6 vs D2	32.5	54.0	6	49	+21.5	[15.0, 28.0]	<0.0001
	D7 vs D0	56.0	52.0	23	15	-4.0	[-10.0, 2.0]	0.2559
	D7 vs D2	32.5	52.0	7	46	+19.5	[13.0, 26.0]	<0.0001
Claude Haiku 5.5	D6 vs D0	50.5	54.0	14	21	+3.5	[-2.0, 9.5]	0.3105
	D6 vs D2	36.5	54.0	7	42	+17.5	[11.0, 24.0]	<0.0001
	D7 vs D0	50.5	55.0	16	25	+4.5	[-2.0, 11.0]	0.2110
	D7 vs D2	36.5	55.0	13	50	+18.5	[11.0, 25.5]	<0.0001
Qwen3-30B-A3B	D6 vs D0	54.5	51.0	23	16	-3.5	[-10.0, 2.5]	0.3368
	D6 vs D2	28.5	51.0	6	51	+22.5	[16.0, 29.5]	<0.0001
	D7 vs D0	54.5	44.5	32	12	-10.0	[-16.5, -3.5]	0.0037
	D7 vs D2	28.5	44.5	14	46	+16.0	[9.0, 23.5]	<0.0001
Gemma 4 26B-A4B	D6 vs D0	72.0	70.0	13	9	-2.0	[-6.5, 2.5]	0.5235
	D6 vs D2	39.0	70.0	9	71	+31.0	[23.5, 39.0]	<0.0001
	D7 vs D0	72.0	64.5	25	10	-7.5	[-13.0, -2.0]	0.0167
	D7 vs D2	39.0	64.5	7	58	+25.5	[18.5, 32.5]	<0.0001

The constraint was read, not merely enforced. A provider may enforce an `enum` during decoding, in which case part of the D6 recovery would belong to the decoder rather than the model. The trajectories allow a partial check. Counting every value a model emitted for the 13 constrained parameters (for the two `pattern` parameters, "off-constraint" means a present list item that fails the regex, not an omitted tag, so the D0 `tags` errors of Table 7, which are omissions, do not appear here), the share outside the constraint is 37 to 69% in D2 for all four models (`ignitionMode="start"`, `order_type="buy"`, `door="all"`) and 0.0% for Luna, Haiku and Gemma 4 in D6, with 3 of 455 values (0.7%) off-constraint for Qwen3. The Qwen3 values show that at least its providers did not enforce the enum, and the fact that the same three models were also at 0.0% in D0, where the constraint existed only as prose that no decoder could enforce, shows that they follow a shown constraint whether or not it is enforced. The check cannot rule out that Azure or Google additionally enforced the schema in D6; what it rules out is that enforcement is needed to explain the recovery. The distinction matters for the recommendation in Section 6, because enforcement is not free: Ray (2026) reports that for a 1.5B-parameter model on a calendar tool-call task, hard schema decoding kept every output schema-valid while executable accuracy fell from 91.5% to 48.0%, so "constraint shown" and "constraint enforced" should be chosen separately.

Two caveats attach to D6. First, it relies on the serving stack passing `enum` and `pattern` through to the model; OpenRouter did so for all four providers here (the schemas the models received are stored with each trajectory), but BFCL itself strips several constraint fields for Gemini-style endpoints and other stacks may do the same. Second, D6 was designed with knowledge of which parameters failed in D2, so it tests the specific hypothesis of Section 5.2, not the general proposition that any schema can be made prose-free.

6 Discussion

The results answer the question of Section 2.3 in a specific way. On BFCL, parameter descriptions are the documentation component the models depend on; the dependence is at the construction decision rather than the selection decision; it is carried by constraints that the schema did not encode, since expressing them structurally restores baseline success with the prose gone; and removing all four carriers removes the ability to do the tasks at all, for every model. These

conclusions rest on paired comparisons over the same 200 tasks, replicate across four models from four providers with different architectures and training regimes, are corroborated by three views of the trajectories (first-call accuracy, earliest divergence and per-parameter error linkage) that do not depend on BFCL's verdict, and in the case of the structural claim were confirmed by a follow-up condition designed to falsify it. The decomposition also says that the carrier of a constraint should not matter, only its presence, and both benchmarks bear this out from opposite sides. D6 moved the constraints from prose into the schema on BFCL and lost nothing; ToolSandbox offers four carriers that restate each other, any one of which can be removed at no cost and all of which cannot. The effect is not a property of a reasoning mode either: it is the same size with reasoning on or off for the three models that can be switched.

The most actionable finding is the pair of Table 7 and Table 9. The descriptions whose removal hurt were not explaining what a parameter means; a model does not need prose to understand that `order_type` is the type of the order. They were carrying an allowed-value list, a format rule or an entity convention that JSON Schema could have expressed as `enum`, `pattern` or a better-chosen name, and that the benchmark's authors happened to put in prose. When those constraints are expressed structurally, every model performs at baseline with no parameter descriptions at all. For tool designers this suggests a simple audit: for every parameter description, ask whether it contains information that is not in the schema's structural fields, and if so move it there, keeping the prose for what structure cannot say. The audit presupposes that the serving stack preserves the structural fields; where it does not, the prose is the only channel and D2 is the relevant condition. The D4 result points the same way from the other side: for three of the four models, replacing a parameter name with `arg_N` while keeping its description cost nothing. Haiku's apparent gain on D4 is diffuse, shrinks to a non-significant $+5$ pp against the repeated D0 run and vanishes with reasoning off, so we treat it as noise; Qwen3's -11.5 pp on D4 is well outside the noise floor and shows that the dependence on names is model-specific. The frontier and open-weight pairs also differ in reasoning (Section 3), so this split cannot be attributed to model family.

The null results for D1 and D3 should not be over-read, and Section 2.3 says why. The tools in BFCL's multi-turn base category have descriptive names (`post_tweet`, `book_flight`), the user prompts name the actions in plain language, and the tool outputs are fully informative; with 18 to 39 tools per task, a model can also distinguish tools by their argument structure. Under those conditions the tool description is redundant with the name and the name is redundant with the parameters, so removing either one leaves the other. D5, where all redundancy is removed at once, shows that the information was being used; it was simply available from more than one place. ToolSandbox, whose 24 to 28 tools include fourteen that take at most one argument, shows the same D1 and D3 nulls for three models and a nominal 6-point D3 loss for Gemma 4, a first sign of the name mattering once argument structure is less distinctive. In settings with ambiguous or overlapping tools, the candidate-set confusion studied by Chattopadhyay et al. (2026) and Rabinovich and Anaby-Tavor (2025), we would expect D1 and D3 to matter more, because there the name and the description would no longer be redundant with the parameter list.

The experiment also asked whether models compensate for missing documentation through interaction, and on this benchmark they do not. The two strategies observed in D5 fail for different reasons. Declining to act (Haiku, Luna) fails immediately because the evaluator requires the state change. Probing (Qwen3, Gemma 4) fails because BFCL caps a turn at 20 steps and scores the final state, so every exploratory call that mutates state is penalised and every one that does not consumes budget. Recovery after an execution error is poor even in D2 (12 to 22%), which suggests that models treat an execution error as a signal to try a different tool or give up rather than to re-read the schema and change the offending argument. Whether a harness that allowed dry runs, more steps or explicit schema introspection would change this is an open question.

Finally, the results say something about benchmarks. The D2 effect flows through a dozen parameters in three backends, six of them dominant (Section 5.2), and disappears when their constraints are written into the schema (Section 5.6). A benchmark that encoded its enums and formats structurally would be more robust to documentation quality and would isolate the models' understanding of task semantics from their ability to parse prose. Conversely, the fact that frontier models fail on `"buy"` versus `"Buy"` when the prose is removed shows how literally stateful benchmarks score: BFCL's single-turn categories compare argument strings after lower-casing and stripping punctuation, but the multi-turn state checker compares backend attributes exactly,[1] so a value the single-turn scorer would accept fails the stateful one. How much of a reported function-calling score rests on such conventions rather than on reasoning is worth knowing for any benchmark.

7 Related work

Teaching language models to call external tools has progressed from self-supervised insertion of API calls (Schick et al., 2023) and reasoning-and-acting prompting (Yao et al., 2023) to instruction-tuned models trained on tens of thousands of real APIs (Patil et al., 2023; Qin et al., 2024), and the schema-based interface of Section 2.1 is now shared by providers and protocols (Anthropic, 2024; OpenAI, 2023). Qu et al. (2025b) survey the area and decompose tool use into task planning, tool selection, tool calling and response generation; the two decisions of Section 2.3 follow their framing, with selection and construction measured separately against ground truth.

On the evaluation side, API-Bank (Li et al., 2023) and the single-turn categories of BFCL (Patil et al., 2025; Yan et al., 2024) score individual calls by abstract-syntax-tree or execution matching. Stateful, multi-turn evaluation was introduced by BFCL V3 (Mao et al., 2024), τ-bench (Yao et al., 2025) and ToolSandbox (Lu et al., 2024), which score a trajectory by comparing the final backend state (and required outputs) with a ground truth rather than by matching individual calls. We use BFCL's multi-turn base category as the main benchmark because its state-based scorer, deterministic backends and per-turn ground truth allow a documentation change to be isolated from every other variable, and ToolSandbox as the second because it differs in all three.

The closest prior work concerns the documentation itself. Hsieh et al. (2023) showed that documentation alone can replace few-shot demonstrations for zero-shot tool use. EasyTool (Yuan et al., 2025) rewrites documentation into concise, standardised instructions, DRAFT (Qu et al., 2025a) refines documentation from execution feedback, and Guo et al. (2026) learn to rewrite descriptions for trace-free deployment, transferring supervision from settings with execution traces to ones without; in an appendix, the last of these report having to correct StableToolBench parameter schemas whose required fields, types and enums did not match the servers, a second instance of a benchmark keeping its constraints somewhere other than the schema. These works treat the documentation as a single object to be improved; we instead ask which component of it carries the value, and find that on a stateful benchmark it is the parameter descriptions, and within those the enums, formats and entity conventions that the schema does not otherwise encode. A small pre-registered ablation on Model Context Protocol tools (Clouâtre, 2026) found that parameter detail can move from the tool-description string into per-parameter schema fields with no measurable loss, the same redundancy between tool-level and parameter-level prose that our D1 result shows.

A second line of work perturbs the interface rather than removing parts of it. Rabinovich

[1] At the pinned commit, `standardize_string` in `bfcl_eval/eval_checker/ast_eval/ast_checker.py` removes spaces and punctuation and lower-cases both sides before the single-turn comparison, whereas `_compare_instances` in `bfcl_eval/eval_checker/multi_turn_eval/multi_turn_checker.py` tests every public attribute of the model's backend instance against the ground-truth instance with Python equality, and the multi-turn response checker likewise matches serialised outputs without normalisation.

and Anaby-Tavor (2025) perturb BFCL queries and expand toolkits with semantically related distractors and find all evaluated models degrade. ToolRobustBench (Zheng et al., 2026) and AgentPerturb (Xu et al., 2026) study perturbations at several stages of a tool-calling pipeline, including rewriting tool descriptions, with mixed conclusions about how damaging interface perturbations are. Chattopadhyay et al. (2026) come closest to our null results from the other direction: they find that minor description rewording and semantics-preserving parameter renames do not meaningfully degrade accuracy, while distractor tools with overlapping descriptions do, which is our D1/D3 finding together with our expectation that D1 and D3 would matter more with overlapping tools. None of these works removes a documentation component outright while holding the rest of the benchmark fixed, which is the manipulation studied here.

A third line concerns the form of the interface. Sigdel and Baral (2026) vary only the form of the tool interface, free-form documentation against JSON Schema against schema plus structured validation diagnostics, with identical tool semantics across conditions, and find in a small pilot that schema conditions reduce interface misuse but not semantic misuse. Their design is the closest to our D6, and their third condition, which feeds the model a structured diagnosis of what was wrong with its call, is the intervention our recovery results leave open. Johnson et al. (2025) argue the opposite direction for tool selection: replacing structured function calling with a natural-language selection step improved accuracy across ten models. The two findings are compatible with ours: structure helps for argument constraints, which is where our effect lives, and may hurt at the selection step, which our ablations barely touch. On the cost side of structure, Ray (2026) shows for small models that enforcing a schema during decoding can keep outputs valid while lowering executable accuracy, so the recommendation of Section 6 is to express constraints in the schema, not necessarily to enforce them.

8 Limitations

The study covers two benchmarks, mostly with one run each. The main results are on BFCL's 200 multi-turn base tasks over eight synthetic backends, run once per condition at temperature 0.001, with a single repeated D0 run for one model; the ToolSandbox control covers 91 scenarios, run once, with a simulated user whose own variation enters every paired difference. The repeat (Section 4.3) shows two identical runs disagreeing on tens of tasks, so differences of a few points, including Haiku's D4 gain, cannot be resolved by this design. The D2, D5 and D6-versus-D2 effects are an order of magnitude larger than the measured floor, and those are the effects on which the conclusions rest.

D5 is not zero semantics. Types, requiredness, enums, response-schema structure, user prompts and tool outputs all remain, as Section 2.1 makes explicit. The D5 collapse measures the joint contribution of the four prose carriers given everything else, not what a model can do with no information. The D5 result is also partly a property of BFCL's evaluator, which caps a turn at 20 steps and scores the final state, so that every exploratory call either consumes budget or mutates state; it should not be read as a claim about what the models could learn with a more permissive harness.

The behavioural metrics are strict. Argument accuracy, exact hits and the first-divergence analysis treat semantically equivalent but differently spelled values as misses. This is applied equally to all conditions but makes the absolute accuracies conservative, and BFCL's own scorer is similarly strict about state. The Qwen3 case study shows a D0/D2 discordance with no documentation cause; the paired statistics account for such noise, but individual discordant tasks should not be read as documentation effects without inspecting the trajectory.

D6 is a targeted test. It was designed after the main analysis, with knowledge of which parameters failed, and it depends on providers passing `enum` and `pattern` through to the model, which they did here. It shows that structure can replace the prose on this benchmark; it does not show how a model would fare on an unfamiliar API whose constraints had never been written

anywhere.

Provider and reasoning settings were not fully controlled. Models were accessed through OpenRouter with default settings. The reasoning-controlled pairs (Section 4.6) set only the on/off switch, not an effort level, and Qwen3 has no such switch, so its reasoning-free result has no reasoning counterpart. Latency is not compared because provider routing differs per model. A third of Qwen3's trajectories had at least one request routed to a second provider, and success on those trajectories is lower in every condition (Section 3), so Qwen3's absolute numbers describe a mixed deployment; its within-model differences are present on both subsets. One Qwen3 task (`multi_turn_base_44`) was rejected with an HTTP 400 by the provider in D0 and again in D6, so its failures are deterministic rather than transient; two other Qwen3 tasks failed once each. All are counted as failures, and dropping them pairwise changes no conclusion.

9 Conclusion

A function-calling schema carries two kinds of information through several carriers at once, and this study asked which carriers a model needs. Holding everything in a stateful multi-turn benchmark fixed except the documentation shown to the model, four models from four providers depend on the same component, parameter descriptions, at the same decision, argument construction, and for the same six parameters plus a small tail of other prose-only enums. What those descriptions supplied was not meaning but constraints: when the same constraints are written into the schema as enums, patterns and precise names, the models perform at baseline with every description blank. Tool names and tool descriptions were free to remove here, and no model compensated for missing documentation by exploring. On a second benchmark, ToolSandbox, where names, descriptions and parameter names all say the same thing, no single one of them could be shown to matter and removing all of them cost 39 to 49 points, so where the semantics live does not matter to these models, only that they are stated somewhere; and switching reasoning on or off moved the level of success but not the effect. The message for tool designers is to put structure in structural fields; the message for benchmark designers is that a function-calling score can rest on documentation conventions that have nothing to do with the task.

Reproducibility statement

The harness, the condition definitions (including the exact schema transformation applied in D6), the analysis code and the paper sources are at https://github.com/Eazydraft/exp-tool-semantics-ablation; BFCL and ToolSandbox are included as submodules pinned to the commits used. Every number, table and figure in this paper is computed from the trajectory files, each of which holds the task, the exact tool schemas sent to the model, the name maps, every raw request and response (with provider and token usage), BFCL's per-turn result and backend state, and the evaluator's verdict. The 6 400 trajectory files of the main run (about 3 GB) are included in the repository under `runs/main/`, together with a small local browser (`./browse.sh`) that shows, for any task and condition, the transcript, the evaluator's verdict with its state diff, and the tool schemas exactly as the model saw them, the 200 files of the repeated Claude Haiku 5.5 baseline run under `runs/main/rerun/`, the 3 600 files of the reasoning-controlled pairs under `runs/main/reasoning/`, and the 2 184 ToolSandbox plays (each with ToolSandbox's own conversation record and execution context next to our trajectory file) under `runs/main/toolsandbox/`; the browser covers both benchmarks and the reasoning variants.

Acknowledgements

This study used the Berkeley Function Calling Leaderboard harness, backends and evaluator unchanged (Gorilla LLM contributors, 2026; Patil et al., 2025); we thank its authors for releasing them, and likewise the authors of ToolSandbox (Lu et al., 2024), used unchanged. Models were accessed through OpenRouter (OpenRouter, 2026).

References

Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, November 2024.

Anthropic. Introducing Claude Haiku 5.5. https://www.anthropic.com/claude-haiku-5-5, October 2026. Announced 2026-10-07. System card: https://www.anthropic.com/document/claude-haiku-5-5-system-card.

Aniruddha Chattopadhyay, Kanishka Halder, and Partha Sarathi Roy. Robust function calling for LLM agents under interface drift, 2026. URL https://arxiv.org/abs/2610.23342.

Hugues Clouâtre. MCP parameter-description ablation: Token savings without accuracy loss. Zenodo, version 1.0.0, September 2026. Pre-registered experiment; code at https://github.com/clouatre-labs/param-description-experiments.

Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961. doi: 10.1080/01621459.1961.10482090.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979. doi: 10.1214/aos/1176344552.

Google DeepMind. Gemma 4: Byte for byte, the most capable open models. https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/, April 2026. Announced 2026-04-02. Model card: https://huggingface.co/google/gemma-4-26b-a4b-it.

Gorilla LLM contributors. Gorilla: Berkeley function calling leaderboard source code. GitHub repository https://github.com/ShishirPatil/gorilla, 2026. Commit `6ea57973`, 2026-03-23, directory `berkeley-function-call-leaderboard`.

Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to rewrite tool descriptions for reliable LLM-agent tool use, 2026. URL https://arxiv.org/abs/2602.20426.

Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Tool documentation enables zero-shot tool-usage with large language models, 2023. URL https://arxiv.org/abs/2308.00675.

Reid T. Johnson, Michelle D. Pain, and Jordan D. West. Natural language tools: A natural language approach to tool calling in large language agents, 2025. URL https://arxiv.org/abs/2510.14453.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3102–3116. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.187.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024. URL https://arxiv.org/abs/2408.04682.

Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Jason Huang, Vishnu Suresh, Yixin Huang, Xiaowen Yu, Joseph E. Gonzalez, and Shishir G. Patil. BFCL V3: Multi-turn & multi-step function calling. Gorilla blog, UC Berkeley, September 2024. URL https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html. Released 2024-09-19, last updated 2024-12-10.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

OpenAI. Function calling and other API updates. https://openai.com/index/function-calling-and-other-api-updates/, June 2023.

OpenAI. Introducing GPT-6 Sol and Luna. https://openai.com/index/introducing-gpt-6-sol-and-luna/, September 2026.

OpenRouter. OpenRouter: A unified interface for LLMs. https://openrouter.ai, 2026. Accessed 2026-11-02.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023. URL https://arxiv.org/abs/2305.15334.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. From exploration to mastery: Enabling LLMs to master tools via self-driven interactions. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025a. URL https://arxiv.org/abs/2410.08197. arXiv:2410.08197.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 2025b. doi: 10.1007/s11704-024-40678-2. URL https://arxiv.org/abs/2405.17935. arXiv:2405.17935.

Ella Rabinovich and Ateret Anaby-Tavor. On the robustness of agentic function calling. In *Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP 2025)*, pages 298–304. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.trustnlp-main.20. arXiv:2504.00914.

Jaideep Ray. The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models, 2026. URL https://arxiv.org/abs/2605.26128.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2302.04761.

Akshey Sigdel and Rista Baral. Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance, 2026. URL https://arxiv.org/abs/2603.13404.

Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON Schema: A media type for describing JSON documents, draft 2020-12. Internet-Draft, https://json-schema.org/draft/2020-12/json-schema-core, 2022.

Yuanjian Xu, Jinyi Han, Jianing Hao, Yuxuan Sun, Fanyang Lu, Richeng Xuan, Guang Zhang, and Zhichao Hu. AgentPerturb: Evaluating LLM agent robustness under controlled interaction perturbations, 2026. URL https://arxiv.org/abs/2610.21624.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. Berkeley function calling leaderboard. Gorilla blog, UC Berkeley, 2024. URL https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html. Last updated 2024-08-19.

An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388. Model card for the July 2025 instruct update: https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2406.12045. arXiv:2406.12045.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Ren Kan, Dongsheng Li, and Deqing Yang. EasyTool: Enhancing LLM-based agents with concise tool instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 951–972. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.naacl-long.44. arXiv:2401.06201.

YiShan Zheng, Yuan Wu, and Yi Chang. ToolRobustBench: Stage-wise perturbation evaluation and failure diagnosis for tool-calling agents, 2026. URL https://arxiv.org/abs/2608.23635.

A Additional tables

Table 10: Documentation sensitivity score $S_m = \frac{1}{N}\sum_i (P_{m,0} - P_{m,i})/P_{m,0}$, the mean relative drop from D0 over the single ablations D1–D4 and over D1–D5. Lower is more robust. The score averages a large D2 loss with near-zero or negative changes elsewhere, so it understates every model's dependence on parameter descriptions; it is reported for completeness only.

Model	S (D1–D4)	S (D1–D5)
GPT-6 Luna	0.103	0.279
Claude Haiku 5.5	0.002	0.202
Qwen3-30B-A3B	0.209	0.367
Gemma 4 26B-A4B	0.118	0.294

Table 11: Earliest divergence among failed tasks (counts). "W.A.": first unmatched ground-truth call was made with wrong arguments; "N.C.": the tool was never called; "None": every ground-truth call was matched but the task still failed.

	D0			D2			D5		
Model	W.A.	N.C.	None	W.A.	N.C.	None	W.A.	N.C.	None
GPT-6 Luna	30	56	2	69	65	1	56	138	4
Claude Haiku 5.5	36	47	16	70	52	5	10	188	2
Qwen3-30B-A3B	44	43	4	95	45	3	78	120	2
Gemma 4 26B-A4B	33	20	3	88	31	3	33	167	0

Table 12: Recovery after an execution error: share of tasks containing at least one failed call that still passed, with the number of such tasks in parentheses.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna	100% (1)	67% (3)	12% (24)	– (0)	33% (3)	0% (38)
Claude Haiku 5.5	50% (2)	– (0)	22% (23)	33% (3)	100% (1)	0% (17)
Qwen3-30B-A3B	25% (4)	0% (4)	20% (30)	8% (12)	0% (5)	0% (102)
Gemma 4 26B-A4B	33% (3)	– (0)	21% (29)	50% (4)	0% (1)	0% (30)

Table 13: Tool-use behaviour, mean per task over all 200 tasks (failures included). Calls are BFCL-decoded tool calls; steps are model requests; a repeated call has identical tool and arguments to an earlier call in the same task; "calls/pass" is the mean number of calls over passing tasks only.

Model	Cond.	calls	steps	on-path	off-path	failed	repeated	distinct tools	calls/pass
GPT-6 Luna	D0	7.4	10.2	5.8	1.6	0.01	0.50	6.2	7.4
	D1	7.5	10.2	5.6	1.9	0.01	0.58	6.0	7.2
	D2	7.6	10.6	5.6	1.9	0.15	0.52	6.0	7.8
	D3	7.6	10.4	5.9	1.7	0.00	0.65	6.2	7.6
	D4	7.7	10.5	5.8	1.8	0.01	0.59	6.3	7.5
	D5	12.9	14.7	4.2	8.4	0.28	0.86	9.1	22.5
Claude Haiku 5.5	D0	7.9	10.1	6.1	1.8	0.01	0.70	6.3	7.6
	D1	7.3	9.7	5.7	1.6	0.00	0.52	6.0	6.9
	D2	8.4	10.6	6.4	1.8	0.12	0.85	6.2	7.8
	D3	7.6	10.0	6.0	1.7	0.01	0.58	6.2	7.4
	D4	8.0	10.2	6.2	1.9	0.01	0.78	6.4	8.0
	D5	4.7	7.3	1.4	3.2	0.10	0.25	3.5	–
Qwen3-30B-A3B	D0	8.0	11.7	6.2	1.9	0.02	0.85	6.5	7.4
	D1	7.8	11.3	5.9	1.8	0.07	0.81	6.3	6.8
	D2	9.4	12.9	6.5	2.4	0.51	1.38	6.6	7.1
	D3	7.9	11.4	6.0	1.8	0.07	0.86	6.4	7.1
	D4	8.3	11.9	6.2	2.1	0.04	1.14	6.4	6.4
	D5	19.5	21.1	4.0	14.4	1.09	3.24	15.6	–
Gemma 4 26B-A4B	D0	8.3	11.4	6.6	1.8	0.02	1.24	6.3	7.4
	D1	8.3	11.5	6.7	1.7	0.00	1.29	6.2	7.4
	D2	11.1	13.8	8.5	2.4	0.23	3.38	5.9	7.2
	D3	8.3	11.4	6.6	1.6	0.02	1.27	6.3	7.4
	D4	8.5	11.6	6.8	1.8	0.01	1.32	6.2	7.8
	D5	11.0	14.0	3.9	6.4	0.73	3.94	5.0	–

Table 14: Terminal error assigned by BFCL per failed task. "State": wrong final state; "Output": required output missing; "Empty": turn ended without a tool call; "Limit": 20-step limit reached; "Provider": inference error from the provider.

Model	Cond.	State	Output	Empty	Limit	Provider	Total failed
GPT-6 Luna	D0	61	17	10	0	0	88
	D1	63	18	14	0	0	95
	D2	96	30	9	0	0	135
	D3	60	15	10	0	0	85
	D4	61	13	9	0	0	83
	D5	93	34	50	21	0	198
Claude Haiku 5.5	D0	64	16	19	0	0	99
	D1	63	15	26	0	0	104
	D2	86	24	17	0	0	127
	D3	57	16	16	0	0	89
	D4	49	9	19	0	0	77
	D5	78	43	78	1	0	200
Qwen3-30B-A3B	D0	64	21	2	3	1	91
	D1	69	27	5	2	0	103
	D2	97	34	6	6	0	143
	D3	64	24	2	4	1	95
	D4	94	14	3	3	0	114
	D5	103	33	5	57	2	200
Gemma 4 26B-A4B	D0	35	15	2	4	0	56
	D1	34	20	4	4	0	62
	D2	76	26	0	20	0	122
	D3	34	17	1	5	0	57
	D4	34	14	0	3	0	51
	D5	94	54	17	35	0	200

B ToolSandbox: setup and results

Setup. ToolSandbox's 1 032 scenarios are 129 base scenarios each in eight variants that vary distraction tools and apply the benchmark's own tool augmentations; we take the 91 base scenarios that need none of the five tools backed by a paid web API, in the variant that offers the agent every tool that the scenario does not itself withhold (24 to 28 tools per scenario after removing those five; 28 in 73 of the 91), so that tool selection is part of every task as it is in BFCL. The user simulator is reached through OpenRouter like the agents, with the same seed in every condition. ToolSandbox executes each call as a Python statement named after the provider's tool-call id; two providers return ids that are not valid Python identifiers (a leading digit, or hyphens), which the unmodified harness turns into a syntax error on every call, so the harness rewrites such ids before execution and keeps them consistent in the history. When a model emits arguments that are not valid JSON, ToolSandbox aborts the scenario; like its own command-line runner we score such a scenario 0 (this happened in 27 of 2 184 plays, all Qwen3's: 26 with invalid JSON arguments, one with a tool name that was not in the list). We report the mean similarity (the benchmark's own metric) and the share of scenarios at similarity 1.0, and test paired differences with a sign-flip permutation test on the similarity and McNemar's test on the binary outcome.

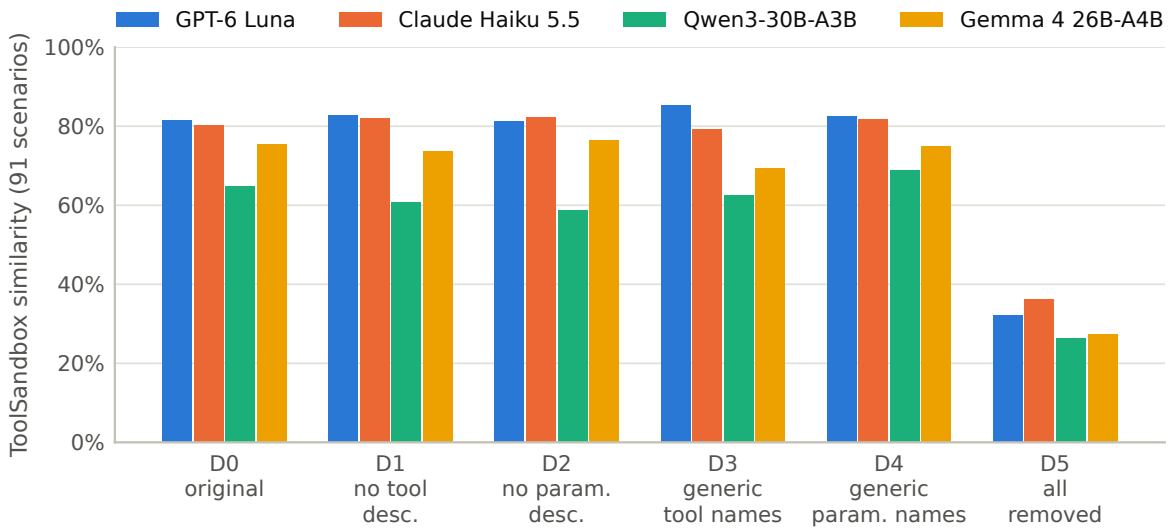


Figure 12: ToolSandbox mean similarity by condition (91 scenarios per bar, same four models). Removing any single component leaves every model within a few points of its baseline; removing all four (D5) costs 39 to 49 points.

Table 15: ToolSandbox results. Left: mean similarity (×100) over the 91 scenarios; right: share of scenarios solved exactly (similarity 1.0). Paired tests for every ablation against D0 are in Table 16, a per-category breakdown in Table 17.

	mean similarity						solved exactly (%)					
Model	D0	D1	D2	D3	D4	D5	D0	D1	D2	D3	D4	D5
GPT-6 Luna	81.5	82.9	81.2	85.4	82.5	32.3	25.3	31.9	25.3	31.9	29.7	14.3
Claude Haiku 5.5	80.1	82.1	82.2	79.3	81.8	36.3	26.4	29.7	25.3	25.3	29.7	17.6
Qwen3-30B-A3B	65.0	60.8	58.7	62.4	69.0	26.3	8.8	14.3	9.9	11.0	14.3	14.3
Gemma 4 26B-A4B	75.5	73.7	76.5	69.5	75.0	27.3	19.8	13.2	19.8	12.1	17.6	16.5

Results. ToolSandbox gives a different answer to the single-component questions and the same answer to the joint one (Figure 12, Table 15). Removing parameter descriptions (D2) changes mean similarity by −0.3, +2.0, −6.3 and +0.9 points for Luna, Haiku, Qwen3 and Gemma 4 (sign-flip $p \geq 0.07$; Table 16), and the share of scenarios solved exactly is unchanged for every model (discordant cells of 5/5, 2/1, 3/4 and 2/2). D1 and D4 are likewise within a few points, and so is D3 except for Gemma 4, which loses 6.0 points of similarity under generic tool names ($p = 0.003$) and seven exactly-solved scenarios against none gained ($p = 0.016$); this is the one single-component effect on ToolSandbox that reaches nominal significance, and it does not reach the corrected threshold. Removing everything (D5) costs 39 to 49 points of similarity for every model ($p < 0.0001$) and is, as on BFCL, an order of magnitude larger than anything a single component does.

The single-component nulls are what the BFCL analysis of Section 5.2 predicts rather than a contradiction of it, because on ToolSandbox every carrier of a tool's semantics restates the others and the schema already encodes every constraint. ToolSandbox's parameter descriptions are generated from docstrings and are short. Across the 72 parameters of the 28 tools an agent can see, none carries an allowed-value list that the schema does not: the only constraints written in prose are formats ("POSIX timestamp", "in 24hr format", "for example celsius"), and most of them restate what the parameter name already says (`reminder_timestamp_lowerbound`, `hour`, `from_unit`). The identifiers that the tools exchange (`person_id`, `reminder_id`) are returned by earlier calls rather than written by the user, so the agent copies them rather than constructs them. There is nothing in these descriptions for D2 to remove that the names, the types and the tool outputs do not supply. On BFCL, the D2 effect flowed through a dozen parameters whose description held the only copy of an enum, a format or an entity type; ToolSandbox has no such parameter and no D2 effect. The one suggestive exception points the same way: Qwen3's −6.3 is concentrated in the 36 *canonicalization* scenarios (70.0 to 58.5; Table 17), in which the user states a date, a duration or a unit in surface form that the agent must convert to a POSIX timestamp or a unit name before calling, that is, in exactly the scenarios where a format convention has to be applied.

Tool selection survives every single-component condition. Taking as reference each model's D0 scenarios with similarity at least 0.9 in which it called a tool (54, 49, 53 and 41 scenarios for Haiku, Gemma 4, Luna and Qwen3), the share in which the ablated run calls every tool that the D0 run used is 0.87, 0.91, 0.91 and 0.91 for Haiku under D1 to D4, 0.86, 0.90, 0.76 and 0.84 for Gemma 4, 0.75, 0.74, 0.75 and 0.83 for Luna and 0.66, 0.76, 0.71 and 0.76 for Qwen3, against 0.07, 0.00, 0.13 and 0.12 under D5. The number of distinct tools called per scenario is between 2.2 and 2.7 for every model in every condition but D5, where it falls to 1.0 for Gemma 4, which mostly stops calling tools, and rises to 5.2 for Qwen3, which probes.

What D5 removes on ToolSandbox is the ability to tell 24 to 28 tools apart by anything but their argument structure, and much of that structure is shared: six tools take no arguments, eight take one, and the four `set_*_status` tools all take a single boolean. The behavioural signature of the collapse matches BFCL's. The number of scenarios in which the agent never calls a tool rises from 6, 3, 7 and 0 (Luna, Haiku, Qwen3, Gemma 4) to 27, 19, 16 and 34; the number that run to the 30-message cap rises from 2 to 23 for Luna and from 10 to 32 for Qwen3; and execution errors per scenario rise three- to five-fold for Luna, Qwen3 and Gemma 4. Two models mostly fall silent (Haiku, Gemma 4) and two mostly probe to the limit (Luna, Qwen3), the same split as in Section 5.3. Because the user is simulated, a silent or flailing agent is also talked at for longer: the user simulator's token usage rises by 40% to 180% in D5.

Two features of ToolSandbox limit what can be read from it. Its score is a similarity rather than a verdict, and the 18 *insufficient-information* scenarios, in which the correct behaviour is to ask rather than act, are scored low for every model in every condition (24 to 74 at D0; Table 17) and move by less than a point under D2. And the user is an LLM: the same user model and the same seed were used in every condition, but the user is not deterministic, so each paired

difference includes user noise as well as agent noise. The sign-flip tests account for this noise; they are not a substitute for a repeated run, which we did not make on ToolSandbox.

Table 16: ToolSandbox: every ablation against D0, paired over the 91 scenarios. Similarity columns are means (×100) with the bootstrap 95% CI of the paired difference and a sign-flip permutation p (10 000 permutations); the exact-match columns give the share of scenarios at similarity 1.0, the discordant cells (solved exactly only in D0 / only in the ablation) and the exact McNemar p.

		mean similarity					solved exactly			
Model	Cond.	D0	abl.	Δ	95% CI	p	D0 %	abl. %	D0/abl. only	McNemar p
GPT-6 Luna	D1	81.5	82.9	+1.4	[-4.5, 7.4]	0.6518	25.3	31.9	5/11	0.2101
	D2	81.5	81.2	-0.3	[-5.7, 5.6]	0.9139	25.3	25.3	5/5	1.0000
	D3	81.5	85.4	+3.9	[-1.9, 9.9]	0.2012	25.3	31.9	5/11	0.2101
	D4	81.5	82.5	+1.0	[-5.0, 7.3]	0.7450	25.3	29.7	5/9	0.4240
	D5	81.5	32.3	-49.2	[-59.0, -39.4]	<0.0001	25.3	14.3	14/4	0.0309
Claude Haiku 5.5	D1	80.1	82.1	+2.0	[-3.4, 7.6]	0.4983	26.4	29.7	1/4	0.3750
	D2	80.1	82.2	+2.0	[-0.8, 5.4]	0.2035	26.4	25.3	2/1	1.0000
	D3	80.1	79.3	-0.8	[-5.0, 3.6]	0.7176	26.4	25.3	2/1	1.0000
	D4	80.1	81.8	+1.6	[-2.2, 6.5]	0.4793	26.4	29.7	1/4	0.3750
	D5	80.1	36.3	-43.8	[-54.8, -32.2]	<0.0001	26.4	17.6	16/8	0.1516
Qwen3-30B-A3B	D1	65.0	60.8	-4.2	[-13.3, 4.4]	0.3525	8.8	14.3	3/8	0.2266
	D2	65.0	58.7	-6.3	[-13.1, 0.4]	0.0706	8.8	9.9	3/4	1.0000
	D3	65.0	62.4	-2.5	[-9.6, 3.9]	0.4796	8.8	11.0	3/5	0.7266
	D4	65.0	69.0	+4.1	[-3.3, 11.7]	0.3038	8.8	14.3	3/8	0.2266
	D5	65.0	26.3	-38.7	[-51.1, -26.2]	<0.0001	8.8	14.3	5/10	0.3018
Gemma 4 26B-A4B	D1	75.5	73.7	-1.8	[-5.1, 1.3]	0.2700	19.8	13.2	6/0	0.0312
	D2	75.5	76.5	+0.9	[-2.9, 4.6]	0.6534	19.8	19.8	2/2	1.0000
	D3	75.5	69.5	-6.0	[-10.4, -2.3]	0.0029	19.8	12.1	7/0	0.0156
	D4	75.5	75.0	-0.5	[-5.8, 4.2]	0.8380	19.8	17.6	5/3	0.7266
	D5	75.5	27.3	-48.2	[-61.2, -34.5]	<0.0001	19.8	16.5	15/12	0.7011

Table 17: ToolSandbox mean similarity (×100) by scenario category, D0 against D2. Categories are ToolSandbox's own and overlap (n is the number of the 91 scenarios in each).

		GPT-6 Luna		Claude Haiku 5.5		Qwen3-30B-A3B		Gemma 4 26B-A4B	
Category	n	D0	D2	D0	D2	D0	D2	D0	D2
single tool call	13	90.2	89.8	92.8	92.7	84.6	79.9	91.3	91.4
multiple tool calls	60	81.9	81.4	86.3	89.3	72.3	64.5	87.6	89.0
single user turn	53	84.0	84.8	88.1	90.6	75.3	67.5	88.4	91.1
multiple user turns	20	81.6	77.8	85.5	88.1	72.3	66.7	87.9	85.0
state dependency	13	74.5	73.3	78.7	86.4	80.3	74.7	87.2	88.7
canonicalization	36	75.8	76.8	88.6	91.8	70.0	58.5	87.4	90.0
insufficient information	18	74.1	74.3	50.7	50.9	26.2	23.9	23.9	24.0

C Case-study trajectories

The four case studies of Section 5.5, selected by the rule given there, with every decoded tool call and its execution result under D0, D2 and D5. Results are truncated to about 100 characters; execution errors are shown in red. The final reply is the model's last plain-text message to the user.

GPT-6 Luna, task `multi_turn_base_177`

Turn 0: Consider this: A patient, a traveler in this context, wants to book a flight using the card with id 'card_7629' and the access token 'abc123xyz', expiring on 10/2026, with CVV 456. They intend

to fly from Los Angeles to Tokyo on December 15, 2026, in business class, estimated at $1,200.

Turn 1: Reflect on the importance of verifying information. Our traveler needs to confirm their flight booking's details for assurance.

Ground truth. t0: `book_flight(access_token='abc123xyz', card_id='card_7629', travel_date='2026-12-15', travel_from='LAX', travel_to='HND', travel_class='business')`; t1: `retrieve_invoice(access_token='abc123xyz', booking_id='3426812')`

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`get_nearest_airport_by_city({"location": "Los Angeles"})`
	`-> {"nearest_airport": "LAX"}`
	`get_nearest_airport_by_city({"location": "Tokyo"})`
	`-> {"nearest_airport": "HND"}`
	`get_flight_cost({"travel_from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
	`-> {"travel_cost_list": [860.0]}`
	`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
	`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`
		`get_nearest_airport_by_city({"location": "Los Angeles"})`
		`-> {"nearest_airport": "LAX"}`
		`get_nearest_airport_by_city({"location": "Tokyo"})`
		`-> {"nearest_airport": "HND"}`
		`get_flight_cost({"travel_from": "Los Angeles", "travel_to": "Tokyo", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> Error during execution: No available route for the given airports.`
		`get_flight_cost({"travel_from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> {"travel_cost_list": [860.0]}`
		`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
		`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`
			(no tool call)

continued on next page

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)
t1	`retrieve_invoice({"access_token": "abc123xyz", "booking_id": "3426812"})` `-> {"invoice": {"booking_id": "3426812", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_...`	`get_booking_history({"access_token": "abc123xyz"})` `-> {"booking_history": {"3426812": {"card_id": "card_7629", "travel_date": "2026-12-15", "travel_fro...`	*(no tool call)*
	Final reply: "I verified the booking invoice. It confirms: - **Route:** Los Angeles (LAX) to Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Fare:** $860 - **Booking ID:** 3426812 - **Transaction ID:** 45451592"	*Final reply:* "I verified the booking against the travel system. Booking **3426812** is listed as: - **Route:** Los Angeles (LAX) → Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Recorded fare:** $860 - **Card:** card_7629 - **Transaction ID:** 45451592 These details match the booking..."	*Final reply:* "Verifying the itinerary before relying on it helps prevent costly mistakes—especially with an international trip. Check the exact travel date, departure and arrival airports, times and time zones, business-class cabin, passenger name, baggage rules, and change or cancellation terms. Confirm that the..."

Claude Haiku 5.5, task `multi_turn_base_78`

Turn 0: I'm gearing up for a quick business getaway and need my ride all set. Would you be able to verify if my tire pressure is in check? If it falls under 37.5 PSI, perhaps we could swing by the nearest tire shop?

Turn 1: While we head over to ensure my tires are roadworthy, I'd like to send out a swift update on my business account. Let's post a tweet: 'Ensuring my wheels are well-maintained. Maintenance is key to success!' with the hashtag 'BusinessOnTheMove'.

Ground truth. t0: `check_tire_pressure()`, `find_nearest_tire_shop()`; t1: `post_tweet(content='Ensuring my wheels are well-maintained. Maintenance is key to success!', tags=['#BusinessOnTheMove'])`

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`check_tire_pressure({})`
	`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
	`find_nearest_tire_shop({})`
	`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
		`check_tire_pressure({})`
		`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
		`find_nearest_tire_shop({})`
		`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
			(no tool call)

continued on next page

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)
t1	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success!", "tags": ["#BusinessOnTheMove"]})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove"})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	*(no tool call)*
	Final reply: "Your tweet is posted on the account @businesspro: "Ensuring my wheels are well-maintained. Maintenance is key to success!" #BusinessOnTheMove I haven't set navigation to the tire shop yet. Do you want me to send you to 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "Your tweet is posted from the businesspro account: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove" Do you want me to set navigation to the tire shop at 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "I can't post that yet. None of my tools are labeled as social media or tweeting functions, so I'd be guessing which one to use, and posting is a public action I don't want to get wrong. Your tweet is ready to go: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTh..."

Qwen3-30B-A3B, task `multi_turn_base_42`

Turn 0: In my existing folder called 'Lectures', I'd like to set up a new file called 'Notes2023.txt'.

Turn 1: Inside 'Notes2023.txt', I shall jot down some pivotal reminders: 'Study diligently, practice programming, master algorithms.'

Turn 2: Ascertain for me how many characters of text remain in this alphabetically sorted version of 'Notes2023.txt'.

Ground truth. t0: `touch(file_name='Notes2023.txt')`; t1: `echo(content='Study diligently, practice programming, master algorithms.', file_name='Notes2023.txt')`; t2: `wc(file_name='Notes2023.txt', mode='c')`

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)

t0	`cd({"folder": "Lectures"})`
	`-> {"error": "cd: 'Lectures': No such file or directory"}`
	
	`ls({})`
	`-> {"current_directory_content": []}`
		`cd({"folder": "Lectures"})`
		`-> {"error": "cd: 'Lectures': No such file or directory"}`
		
		`ls({})`
		`-> {"current_directory_content": []}`
			`cat({"file_name": "Lectures/Notes2023.txt"})`
			`-> {"error": "cat: 'Lectures/Notes2023.txt': Invalid character"}`
			
			`cd({"folder": "Lectures/Notes2023.txt"})`
			`-> {"error": "cd: Lectures/Notes2023.txt: Unsupported path. Only one folder level at a time is suppo...`

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)
	`pwd({})` `-> {"current_working_directory": "/Lectures"}`		`cp({"source": "Lectures", "destination": "exists"})` `-> {"error": "cp: cannot copy 'Lectures': No such file or directory"}`
	`ls({})` `-> {"current_directory_content": []}`		`diff({"file_name1": "Lectures", "file_name2": "create"})` `-> {"error": "diff: Lectures or create: No such file or directory"}`
	`touch({"file_name": "Notes2023.txt"})` `-> None`		`du({})` `-> {"disk_usage": "0 bytes"}`
			`echo({"content": "Lectures"})` `-> {"terminal_output": "Lectures"}`
			`find({"path": "Lectures/Notes2023.txt"})` `-> {"error": "find: 'Lectures/Notes2023.txt': No such file or directory"}`
			`grep({"file_name": "Lectures", "pattern": "list"})` `-> {"error": "grep: Lectures: No such file or directory"}`
			`ls({})` `-> {"current_directory_content": []}`
			`mkdir({"dir_name": "Lectures"})` `-> None`
			`mv({"source": "Notes2023.txt", "destination": "create"})` `-> {"error": "mv: cannot move 'Notes2023.txt': No such file or directory"}`
t1	`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})` `-> None`	`mkdir({"dir_name": "Lectures"})` `-> None`	*(no tool call)*
		`cd({"folder": "Lectures"})` `-> {"current_working_directory": "Lectures"}`	

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)

		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
		`-> {"error": "echo: cannot write to 'Notes2023.txt': No such file"}`
		`touch({"file_name": "Notes2023.txt"})`
		`-> None`
		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
		`-> None`	

t2	`sort({"file_name": "Notes2023.txt"})`
	`-> {"sorted_content": "Study diligently, practice programming, master algorithms."}`
	`wc({"file_name": "Notes2023.txt", "mode": "c"})`
	`-> {"count": 58, "type": "characters"}`
		`sort({"file_name": "Notes2023.txt"})`
		`-> {"sorted_content": "Study diligently, practice programming, master algorithms."}`
		`wc({"file_name": "Notes2023.txt", "mode": "c"})`
		`-> {"count": 58, "type": "characters"}`
			(no tool call)

	Final reply: "The alphabetically sorted version of 'Notes2023.txt' contains 58 characters. This includes all letters, spaces, and punctuation marks in the text "Study diligently, practice programming, master algorithms."""	*Final reply:* "I've analyzed the 'Notes2023.txt' file and found the following: The file contains 58 characters in total (including spaces and punctuation). When sorted alphabetically, the content remains the same: "Study diligently, practice programming, master algorithms." This is because the text is already in..."	*Final reply:* "It seems that the file 'Notes2023.txt' could not be created or accessed due to system limitations. As a result, there is no content to analyze or count characters from. If you'd like, I can help you draft the text you want to include in the file, or guide you through creating it in a different envi..."

Gemma 4 26B-A4B, task `multi_turn_base_11`

Turn 0: Display all the available files located within the '/temp' directory including hidden one using the terminal interface .

Turn 1: With the information presented on the terminal, draft a post on Twitter mentioning only the file names seperated them by comma and space, and include the hashtag '#fileshowcase' to spread the word.

Ground truth. t0: `ls(a=True)`; t1: `post_tweet(content='file1.txt, file2.txt', tags=['#fileshowcase'])`

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)

t0	`cd({"folder": "temp"})`
	`-> {"error": "cd: 'temp': No such file or directory"}`
	
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	
	`find({"path": "/temp"})`
	`-> {"error": "find: '/temp': No such file or directory"}`
	
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	
	`find({"name": "temp", "path": "/"})`
	`-> {"matches": []}`
	
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		
		`cd({"folder": "/temp"})`
		`-> {"error": "cd: /temp: Unsupported path. Only one folder level at a time is supported."}`
		
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		
		`find({"path": "/temp"})`
		`-> {"error": "find: '/temp': No such file or directory"}`
		
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		
		`find({"path": "."})`
		`-> {"matches": ["./file1.txt", "./file2.txt"]}`
		
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		
		`find({"path": "/"})`
		`-> {"matches": ["/file1.txt", "/file2.txt"]}`
		
		`find({"path": "/"})`
		`-> {"matches": ["/file1.txt", "/file2.txt"]}`
		
		`find({"path": "/temp"})`
		`-> {"error": "find: '/temp': No such file or directory"}`
			`get_user_stats({"username": "ls -a /temp"})`
			`-> {"tweet_count": 0, "following_count": 0, "retweet_count": 0}`

continued on next page

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)
t1	`post_tweet({"content": "file1.txt, file2.txt", "tags": ["#fileshowcase"]})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt", "tags": ["#fileshowcase"], "m...`	`post_tweet({"content": "file1.txt, file2.txt #fileshowcase"})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt #fileshowcase", "tags": [], "me...`	*(no tool call)*
	Final reply: "I have posted the tweet for you: "file1.txt, file2.txt #fileshowcase"."	*Final reply:* "The tweet has been posted successfully: "file1.txt, file2.txt #fileshowcase""	*Final reply:* "Since no files were found in the '/temp' directory, I cannot list any file names. If you can provide the file names or if they appear after a different command, I will be happy to draft the tweet for you!"